

Function Approximations: Reinforcement Learning - Lecture 11

Felipe Meneguzzi
`felipe.meneguzzi@pucrs.br`

PUCRS

Material adapted from:
Martha and Adam White - University of Alberta/Coursera
Sutton and Barto - Reinforcement Learning: An Introduction (2nd ed.)
David Silver - Reinforcement Learning

Outline

1 Recap

2 Introduction

3 Incremental Methods

Gradient Descent

Linear Function Approximation

Incremental Prediction Algorithms

Incremental Control Algorithms

Coarse Coding and Non-linear FA

Mountain Car

Convergence

4 Batch Methods

Least Squares Prediction

Least Squares Control

Exploration under Function Approximation

5 Summary and Conclusion

- Last lecture:
 - Integrating Planning and Learning
 - Dyna-Q
- This lecture (Sutton and Barto 9.1-9.5, 10.1-10.2)
 - **Function approximations**
 - Approximation-based prediction and Control
- Next lecture
 - Policy Gradient Methods
 - Actor-Critic architectures

Outline

- ① Recap
- ② Introduction
- ③ Incremental Methods
 - Gradient Descent
 - Linear Function Approximation
 - Incremental Prediction Algorithms
 - Incremental Control Algorithms
 - Coarse Coding and Non-linear FA
 - Mountain Car
 - Convergence
- ④ Batch Methods
 - Least Squares Prediction
 - Least Squares Control
 - Exploration under Function Approximation
- ⑤ Summary and Conclusion

Large-Scale Reinforcement Learning

Reinforcement learning can be used to solve **large** problems, e.g.

- Backgammon: 10^{20} states
- Computer Go: 10^{170} states
- Helicopter: continuous state space

Large-Scale Reinforcement Learning

Reinforcement learning can be used to solve **large** problems, e.g.

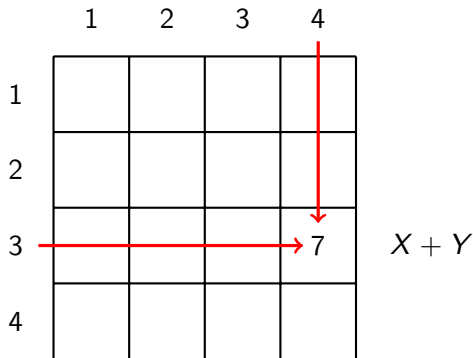
- Backgammon: 10^{20} states
- Computer Go: 10^{170} states
- Helicopter: continuous state space

How can we scale up the model-free methods for prediction and control from the previous lectures?

Value Function Representation

A value function can be represented in different ways

State	Value
S_1	-4
S_2	6
S_3	12
S_4	5
S_5	53
...	
S_{16}	-9



Value Function Approximation

- So far we have represented value function by a lookup table
 - Every state s has an entry $V(s)$
 - Or every state-action pair s, a has an entry $Q(s, a)$

Value Function Approximation

- So far we have represented value function by a lookup table
 - Every state s has an entry $V(s)$
 - Or every state-action pair s, a has an entry $Q(s, a)$
- Problem with large MDPs:
 - There are too many states and/or actions to store in memory
 - It is too slow to learn the value of each state individually

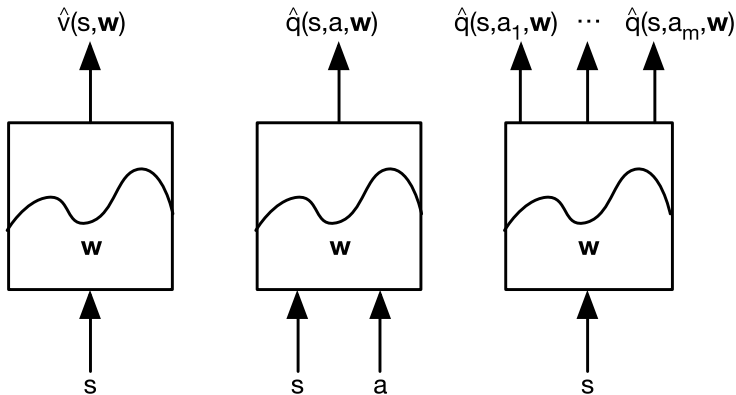
Value Function Approximation

- So far we have represented value function by a lookup table
 - Every state s has an entry $V(s)$
 - Or every state-action pair s, a has an entry $Q(s, a)$
- Problem with large MDPs:
 - There are too many states and/or actions to store in memory
 - It is too slow to learn the value of each state individually
- Solution for large MDPs:
 - Estimate value function with **function approximation**

$$\hat{v}(s, \mathbf{w}) \approx v_{\pi}(s)$$
$$\text{or } \hat{q}(s, a, \mathbf{w}) \approx q_{\pi}(s, a)$$

- Generalize from seen states to unseen states
- Update parameter $\mathbf{w}$ using MC or TD learning

Types of Value Function Approximation



Which Function Approximator?

There are many function approximators, e.g.

- Linear combinations of features
- Neural network
- Decision tree
- Nearest neighbor
- Fourier / wavelet bases
- ...

Which Function Approximator?

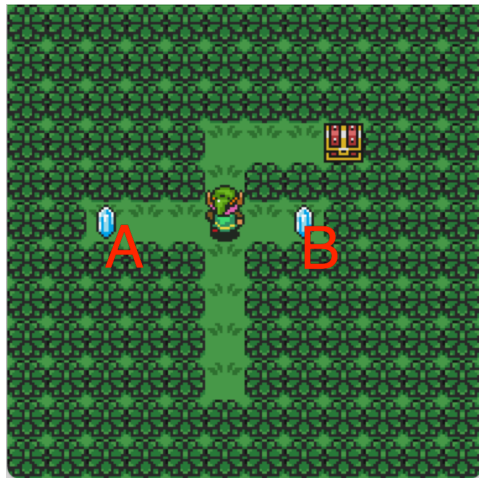
We consider **differentiable** function approximators, e.g.

- **Linear combinations of features**
- **Neural network**
- Decision tree
- Nearest neighbor
- Fourier / wavelet bases ...

Furthermore, we require a training method that is suitable for **non-stationary, non-iid** data

Why parameterized value functions?

- Parameterized value functions help generalize experience
 - When Link visits A, he should immediately know that B is also valuable
 - Learn values for states without visiting them
- Attributes here: $(x, y, \text{rupee}, \text{treasure})$



Parameterizing the Value Function

$$\hat{v}(s, \underbrace{w}_{\text{weights}}) \approx v_{\pi}(s)$$

Example of Parameterized Value Function

$$\hat{v}(s, \mathbf{w}) \doteq w_1 X + w_2 Y$$

	1	2	3	4
1				
2				
3				
4				

Example of Parameterized Value Function

$$\hat{v}(s, \mathbf{w}) \doteq w_1 X + w_2 Y$$

We only have to store the two weights

	1	2	3	4
1				
2				
3				
4				

Example of Parameterized Value Function

$$\hat{v}(s, \mathbf{w}) \doteq w_1 X + w_2 Y$$

We only have to store the two weights

$$w_1 = 1$$

$$w_2 = 1$$

	1	2	3	4
1	2	3	4	5
2	3	4	5	6
3	4	5	6	7
4	5	6	7	8

Example of Parameterized Value Function

$$\hat{v}(s, \mathbf{w}) \doteq w_1 X + w_2 Y$$

We only have to store the two weights

$$w_1 = 2$$

$$w_2 = 1$$

	1	2	3	4
1	3	5	7	9
2	4	6	8	10
3	5	7	9	11
4	6	8	10	12

Example of Parameterized Value Function

$$\hat{v}(s, \mathbf{w}) \doteq w_1 X + w_2 Y$$

We only have to store the two weights

$$w_1 = -1$$

$$w_2 = 1$$

	1	2	3	4
1	0	-1	-2	-3
2	1	0	-1	-2
3	2	1	0	-1
4	3	2	1	0

Example of Parameterized Value Function

$$\hat{v}(s, \mathbf{w}) \doteq w_1 X + w_2 Y$$

We only have to store the two weights

$$w_1 = 4$$

$$w_2 = 1$$

	1	2	3	4
1	5	9	13	17
2	6	10	14	18
3	7	11	15	19
4	8	12	16	20

Limitations of Linear Approximation

Linear Value Function Approximation

$$\hat{v}(s, \mathbf{w}) \doteq \sum w_i \mathbf{x}_i(s)$$

	1	2	3	4
1				
2				
3				
4				

Limitations of Linear Approximation

Linear Value Function Approximation

$$\hat{v}(s, \mathbf{w}) \doteq \sum w_i \mathbf{x}_i(s)$$

	1	2	3	4
1	0	0	0	0
2	0	5	5	0
3	0	5	5	0
4	0	0	0	0

Tabular Value functions are linear functions

A value function can be represented in different ways

State	Value
S_1	
S_2	
S_3	
S_4	
S_5	
$\dots$	
S_i	
$\dots$	
S_{16}	

Tabular Value functions are linear functions

A value function can be represented in different ways

State	Value
S_1	
S_2	
S_3	
S_4	
S_5	
...	
S_i	
...	
S_{16}	

Tabular Value functions are linear functions

A value function can be represented in different ways

State	Value
s_1	
s_2	
s_3	
s_4	
s_5	
...	
s_i	
...	
s_{16}	

$$\hat{v}(s_i, \mathbf{w}) \doteq \langle \mathbf{w}, \mathbf{x}(s_i) \rangle$$

Tabular Value functions are linear functions

A value function can be represented in different ways

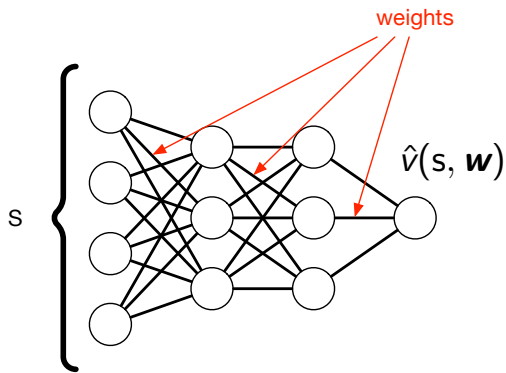
State	Value
S_1	
S_2	
S_3	
S_4	
S_5	
$\dots$	
S_i	
$\dots$	
S_{16}	

$$\hat{v}(s_i, \mathbf{w}) \doteq \langle \mathbf{w}, \mathbf{x}(s_i) \rangle$$
$$= w_i$$

$\mathbf{x}(s_i) =$
 $i^{th} \text{ element}$

$$\begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

Nonlinear Function Approximation



Neural Network

Outline

1 Recap

2 Introduction

3 Incremental Methods

Gradient Descent

Linear Function Approximation

Incremental Prediction Algorithms

Incremental Control Algorithms

Coarse Coding and Non-linear FA

Mountain Car

Convergence

4 Batch Methods

Least Squares Prediction

Least Squares Control

Exploration under Function Approximation

5 Summary and Conclusion

Introducing Gradient Descent

Gradient: Derivatives in Multiple Dimensions

$$\mathbf{w} \doteq \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_d \end{bmatrix}$$

$$\nabla f \doteq \begin{bmatrix} \frac{\partial f}{\partial w_1} \\ \frac{\partial f}{\partial w_2} \\ \vdots \\ \frac{\partial f}{\partial w_d} \end{bmatrix}$$

- $\frac{\partial f}{\partial w_2}$ is the direction to change w_2 in order to increase f
- How quickly f changes
- The gradient gives the direction of steepest ascent

Gradient Descent

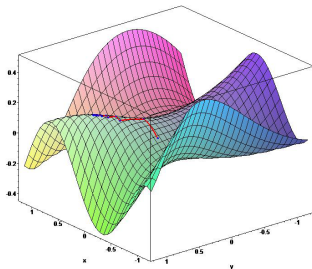
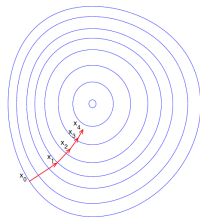
- Let $J(\mathbf{w})$ be a differentiable function of parameter vector $\mathbf{w}$
- Define the gradient of $J(\mathbf{w})$ to be:

$$\nabla_{\mathbf{w}} J(\mathbf{w}) \doteq \left(\frac{\partial J(\mathbf{w})}{\partial w_1} \quad \dots \quad \frac{\partial J(\mathbf{w})}{\partial w_n} \right)^\top = \begin{pmatrix} \frac{\partial J(\mathbf{w})}{\partial w_1} \\ \vdots \\ \frac{\partial J(\mathbf{w})}{\partial w_n} \end{pmatrix}$$

- To find a local minimum of $J(\mathbf{w})$
- Adjust $\mathbf{w}$ in direction of $-ve$ gradient

$$\Delta \mathbf{w} = -\frac{1}{2} \alpha \nabla_{\mathbf{w}} J(\mathbf{w})$$

where α is a step-size parameter



Value Function Approx. By Stochastic Gradient Descent

- Goal: find parameter vector $\mathbf{w}$ minimizing mean-squared error between approximate value fn $\hat{v}(s, \mathbf{w})$ and true value fn $v_\pi(s)$

$$J(\mathbf{w}) = \mathbb{E}_\pi \left[(v_\pi(S) - \hat{v}(S, \mathbf{w}))^2 \right]$$

- Gradient descent finds a local minimum

$$\begin{aligned} \Delta \mathbf{w} &= \frac{1}{2} \alpha \nabla_{\mathbf{w}} J(\mathbf{w}) \\ &= \alpha \mathbb{E}_\pi [(v_\pi(S) - \hat{v}(S, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S, \mathbf{w})] \end{aligned}$$

- Stochastic gradient descent samples the gradient

$$\Delta \mathbf{w} = \alpha (v_\pi(S) - \hat{v}(S, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S, \mathbf{w})$$

- Expected update is equal to full gradient update

Value Function Approx. By Stochastic Gradient Descent

- Goal: find parameter vector $\mathbf{w}$ minimizing mean-squared error between approximate value fn $\hat{v}(s, \mathbf{w})$ and true value fn $v_\pi(s)$

$$J(\mathbf{w}) = \mathbb{E}_\pi \left[(v_\pi(S) - \hat{v}(S, \mathbf{w}))^2 \right]$$

- Gradient descent finds a local minimum

$$\begin{aligned} \Delta \mathbf{w} &= \frac{1}{2} \alpha \nabla_{\mathbf{w}} J(\mathbf{w}) \\ &= \alpha \mathbb{E}_\pi [(v_\pi(S) - \hat{v}(S, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S, \mathbf{w})] \end{aligned}$$

- Stochastic gradient descent samples the gradient

$$\Delta \mathbf{w} = \alpha (v_\pi(S) - \hat{v}(S, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S, \mathbf{w})$$

- Expected update is equal to full gradient update

Recall that $v_\pi(s)$ is an oracle!

Feature Vectors

- Represent state by a **feature vector**

$$\mathbf{x}(S) = \begin{pmatrix} \mathbf{x}_1(S) \\ \vdots \\ \mathbf{x}_n(S) \end{pmatrix}$$

- For example:
 - Distance of robot from landmarks
 - Trends in the stock market
 - Piece and pawn configurations in chess

Linear Value Function Approximation

- Represent value function by a linear combination of features

$$\hat{v}(S, \mathbf{w}) = \mathbf{x}(S)^\top \mathbf{w} = \sum_{j=1}^n \mathbf{x}_j(S) \mathbf{w}_j$$

- Objective function is quadratic in parameters $\mathbf{w}$

$$J(\mathbf{w}) = \mathbb{E}_\pi \left[(v_\pi(S) - \mathbf{x}(S)^\top \mathbf{w})^2 \right]$$

- Stochastic gradient descent converges on **global** optimum
- Update rule is particularly simple

$$\begin{aligned} \nabla_{\mathbf{w}} \hat{v}(S, \mathbf{w}) &= \mathbf{x}(S) \\ \Delta \mathbf{w} &= \alpha (v_\pi(S) - \hat{v}(S, \mathbf{w})) \mathbf{x}(S) \end{aligned}$$

Update = **step-size** $\times$ **prediction error** $\times$ **feature value**

Table Lookup Features

- Table lookup is a special case of linear value function approximation
- Using **table lookup features**

$$\mathbf{x}^{table}(S) = \begin{pmatrix} 1(S = s_1) \\ \vdots \\ 1(S = s_n) \end{pmatrix}$$

- Parameter vector $\mathbf{w}$ gives value of each individual state

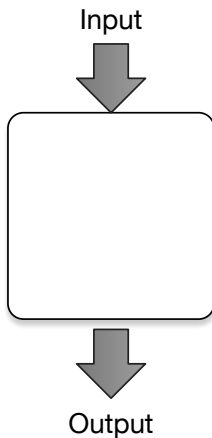
$$\hat{v}(S, \mathbf{w}) = \begin{pmatrix} 1(S = s_1) \\ \vdots \\ 1(S = s_n) \end{pmatrix} \cdot \begin{pmatrix} \mathbf{w}_1 \\ \vdots \\ \mathbf{w}_n \end{pmatrix}$$

Supervised Learning

$\{((2 \text{ bed}, 103m^2, \dots), \$300000)$
 $((1 \text{ bed}, 80m^2, \dots), \$170000)$
 $\dots$
 $((2 \text{ bed}, 103m^2, \dots), \$515000) \}$

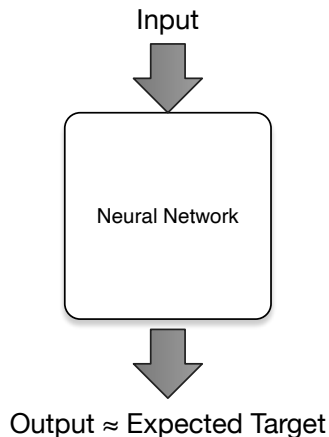
Supervised Learning

$\{((2 \text{ bed}, 103m^2, \dots), \$300000)$
 $((1 \text{ bed}, 80m^2, \dots), \$170000)$
 $\dots$
 $((2 \text{ bed}, 103m^2, \dots), \$515000) \}$



Supervised Learning

$\{((2 \text{ bed}, 103m^2, \dots), \$300000)$
 $((1 \text{ bed}, 80m^2, \dots), \$170000)$
 $\dots$
 $((2 \text{ bed}, 103m^2, \dots), \$515000) \}$



Framing Value Estimation as Supervised Learning

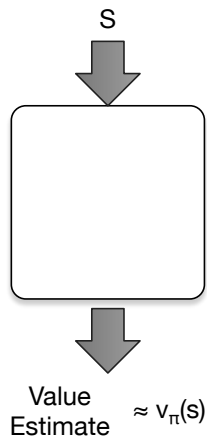
Monte Carlo

$$\{(S_1, G_1)$$
$$(S_2, G_2)$$
$$(S_3, G_3)$$
$$\dots\}$$

Framing Value Estimation as Supervised Learning

Monte Carlo

$\{(S_1, G_1)$
 (S_2, G_2)
 (S_3, G_3)
 $\dots\}$



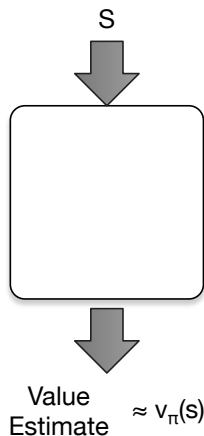
Framing Value Estimation as Supervised Learning

Monte Carlo

$\{(S_1, G_1)$
 (S_2, G_2)
 (S_3, G_3)
 $\dots\}$

TD

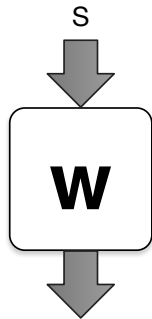
$\{(S_1, R_2 + \gamma \hat{v}(S_2, \mathbf{w}))$
 $(S_2, R_3 + \gamma \hat{v}(S_3, \mathbf{w}))$
 $(S_3, R_4 + \gamma \hat{v}(S_4, \mathbf{w}))$
 $\dots\}$



The Value Error Objective

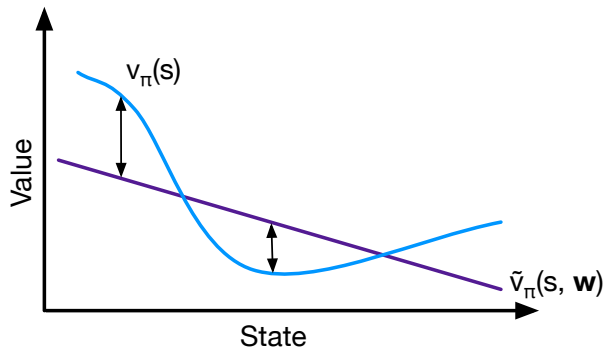
Idealized scenario:

$$\{(S_1, v_\pi(S_1)), (S_2, v_\pi(S_2)), (S_3, v_\pi(S_3)), \dots\}$$

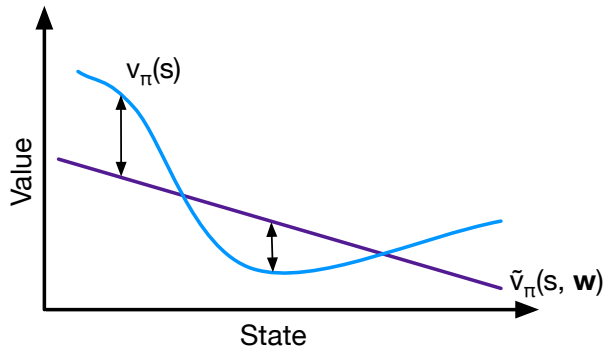


$$\tilde{v}_\pi(s, \mathbf{w}) \approx v_\pi(s)$$

The Mean Squared Error Objective

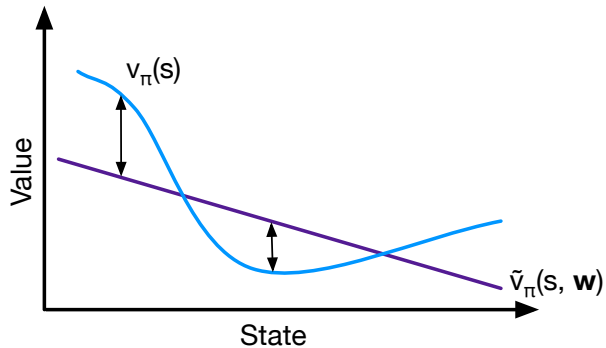


The Mean Squared Error Objective



$$[v_\pi(s) - \hat{v}(s, \mathbf{w})]^2$$

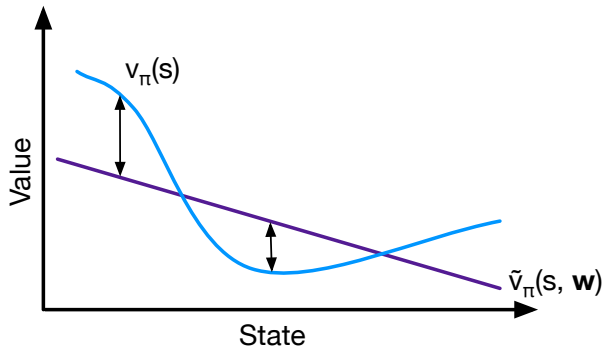
The Mean Squared Error Objective



$$\mu(s) [v_\pi(s) - \hat{v}(s, \mathbf{w})]^2$$

The Mean Squared Error Objective

Mean Squared Value Error



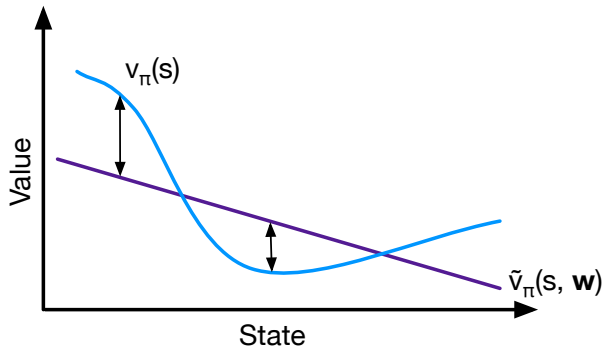
$$\sum_s \mu(s) [v_{\pi}(s) - \hat{v}(s, \mathbf{w})]^2$$

The Mean Squared Error Objective

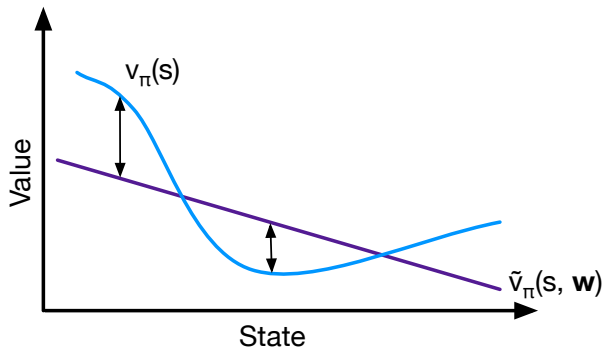
Mean Squared Value Error

$$\sum_s \mu(s) [v_\pi(s) - \hat{v}(s, \mathbf{w})]^2$$

$\mu(s)$ is the fraction of time we spend in s , when following π



The Mean Squared Error Objective



Adapting the weights to minimize the Mean Squared Value Error Objective

$$\overline{VE} = \sum_s \mu(s) [v_{\pi}(s) - \hat{v}(s, \mathbf{w})]^2$$

Gradient Monte Carlo for Policy Evaluation

$$\begin{aligned} & \nabla \sum_{s \in \mathcal{S}} \mu(s) [v_{\pi}(s) - \hat{v}(s, \mathbf{w})]^2 \\ &= \sum_{s \in \mathcal{S}} \mu(s) \nabla [v_{\pi}(s) - \hat{v}(s, \mathbf{w})]^2 \\ &= - \sum_{s \in \mathcal{S}} \mu(s) 2 [v_{\pi}(s) - \hat{v}(s, \mathbf{w})] \nabla \hat{v}(s, \mathbf{w}) \end{aligned}$$

Gradient Monte Carlo for Policy Evaluation

$$\begin{aligned} & \nabla \sum_{s \in \mathcal{S}} \mu(s) [v_{\pi}(s) - \hat{v}(s, \mathbf{w})]^2 & \hat{v}(s, \mathbf{w}) & \doteq \langle \mathbf{w}, \mathbf{x}(s) \rangle \\ &= \sum_{s \in \mathcal{S}} \mu(s) \nabla [v_{\pi}(s) - \hat{v}(s, \mathbf{w})]^2 \\ &= - \sum_{s \in \mathcal{S}} \mu(s) 2 [v_{\pi}(s) - \hat{v}(s, \mathbf{w})] \nabla \hat{v}(s, \mathbf{w}) \end{aligned}$$

Gradient Monte Carlo for Policy Evaluation

$$\begin{aligned} & \nabla \sum_{s \in \mathcal{S}} \mu(s) [v_{\pi}(s) - \hat{v}(s, \mathbf{w})]^2 \\ &= \sum_{s \in \mathcal{S}} \mu(s) \nabla [v_{\pi}(s) - \hat{v}(s, \mathbf{w})]^2 \\ &= - \sum_{s \in \mathcal{S}} \mu(s) 2 [v_{\pi}(s) - \hat{v}(s, \mathbf{w})] \nabla \hat{v}(s, \mathbf{w}) \end{aligned}$$

$$\hat{v}(s, \mathbf{w}) \doteq \langle \mathbf{w}, \mathbf{x}(s) \rangle$$

$$\nabla \hat{v}(s, \mathbf{w}) = \mathbf{x}(s)$$

Gradient Monte Carlo for Policy Evaluation

$$\begin{aligned} & \nabla \sum_{s \in \mathcal{S}} \mu(s) [v_{\pi}(s) - \hat{v}(s, \mathbf{w})]^2 \\ &= \sum_{s \in \mathcal{S}} \mu(s) \nabla [v_{\pi}(s) - \hat{v}(s, \mathbf{w})]^2 \\ &= - \sum_{s \in \mathcal{S}} \mu(s) 2 [v_{\pi}(s) - \hat{v}(s, \mathbf{w})] \nabla \hat{v}(s, \mathbf{w}) \end{aligned}$$

$$\begin{aligned} \hat{v}(s, \mathbf{w}) &\doteq \langle \mathbf{w}, \mathbf{x}(s) \rangle \\ \nabla \hat{v}(s, \mathbf{w}) &= \mathbf{x}(s) \end{aligned}$$

$$\Delta \mathbf{w} \propto \sum_{s \in \mathcal{S}} \mu(s) [v_{\pi}(s) - \hat{v}(s, \mathbf{w})] \nabla \hat{v}(s, \mathbf{w})$$

From Gradient Descent to Stochastic Gradient Descent

$$\sum_{s \in \mathcal{S}} \mu(s)^2 [v_\pi(s) - \hat{v}(s, \mathbf{w})] \nabla \hat{v}(s, \mathbf{w})$$

$$\{(S_1, v_\pi(S_1)), (S_2, v_\pi(S_2)), (S_3, v_\pi(S_3)), \dots\}$$

From Gradient Descent to Stochastic Gradient Descent

$$\sum_{s \in \mathcal{S}} \mu(s)^2 [v_\pi(s) - \hat{v}(s, \mathbf{w})] \nabla \hat{v}(s, \mathbf{w})$$

$$\left\{ \boxed{(S_1, v_\pi(S_1))}, (S_2, v_\pi(S_2)), (S_3, v_\pi(S_3)), \dots \right\}$$

$$\mathbf{w}_2 \doteq \mathbf{w}_1 + \alpha [v_\pi(S_1) - \hat{v}(S_1, \mathbf{w}_1)] \nabla \hat{v}(S_1, \mathbf{w}_1)$$

From Gradient Descent to Stochastic Gradient Descent

$$\sum_{s \in \mathcal{S}} \mu(s)^2 [v_\pi(s) - \hat{v}(s, \mathbf{w})] \nabla \hat{v}(s, \mathbf{w})$$

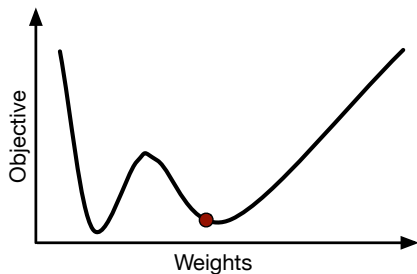
$$\left\{ (S_1, v_\pi(S_1)), \boxed{(S_2, v_\pi(S_2))}, (S_3, v_\pi(S_3)), \dots \right\}$$

$$\mathbf{w}_3 \doteq \mathbf{w}_2 + \alpha [v_\pi(S_2) - \hat{v}(S_2, \mathbf{w}_2)] \nabla \hat{v}(S_2, \mathbf{w}_2)$$

From Gradient Descent to Stochastic Gradient Descent

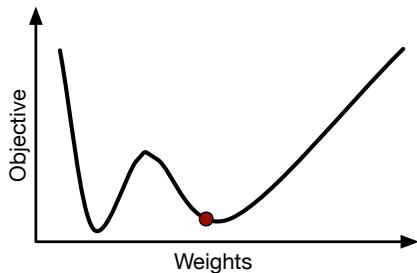
$$\sum_{s \in \mathcal{S}} \mu(s)^2 [v_\pi(s) - \hat{v}(s, \mathbf{w})] \nabla \hat{v}(s, \mathbf{w})$$

$$\{(S_1, v_\pi(S_1)), (S_2, v_\pi(S_2)), (S_3, v_\pi(S_3)), \dots\}$$



From Gradient Descent to Stochastic Gradient Descent

$$\sum_{s \in \mathcal{S}} \mu(s) 2 [v_{\pi}(s) - \hat{v}(s, \mathbf{w})] \nabla \hat{v}(s, \mathbf{w})$$



For Monte Carlo, you can drop the oracle $v_{\pi}(s)$ and rely entirely on sampling.

Incremental Prediction Algorithms

- We have assumed true value function $v_\pi(s)$ given by supervisor
- But in RL there is no supervisor, only rewards
- In practice, we substitute a target for $v_\pi(s)$:
 - For MC, the target is the return G_t

$$\Delta \mathbf{w} = \alpha(\mathbf{G}_t - \hat{v}(S_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w})$$

- For TD(0), the target is the TD target $R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w})$

$$\Delta \mathbf{w} = \alpha(\mathbf{R}_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}) - \hat{v}(S_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w})$$

- For TD(λ), the target is the λ -return G_t^λ

$$\Delta \mathbf{w} = \alpha(\mathbf{G}_t^\lambda - \hat{v}(S_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w})$$

Monte-Carlo with Value Function Approximation

- Return G_t is an unbiased, noisy sample of true value $v_\pi(s_t)$
- Can therefore apply supervised learning to “training data”:

$$\langle S_1, G_1 \rangle, \langle S_2, G_2 \rangle, \dots, \langle S_T, G_T \rangle$$

- For example, using **linear Monte-Carlo policy evaluation**

$$\begin{aligned}\Delta \mathbf{w} &= \alpha(\mathbf{G}_t - \hat{v}(S_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}) \\ &= \alpha(\mathbf{G}_t - \hat{v}(S_t, \mathbf{w})) \mathbf{x}(S_t)\end{aligned}$$

- Monte-Carlo evaluation converges to a local optimum
- Even when using non-linear value function approximation

Gradient Monte Carlo

Gradient MC for estimating $\hat{V} \approx v_\pi$

Require: the policy π to be evaluated

Require: a differentiable function $\hat{v} : \mathcal{S} \times \mathbb{R}^d \mapsto \mathbb{R}$

Require: a step size $\alpha > 0$

- 1: **function** approxMC($\alpha, \pi, \hat{v}$)
- 2: Initialize value-function weights $\mathbf{w} \in \mathbb{R}$ arbitrarily
- 3: **loop** forever (for each episode)
- 4: Generate an episode following π : $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$

Gradient Monte Carlo

Gradient MC for estimating $\hat{V} \approx v_\pi$

Require: the policy π to be evaluated

Require: a differentiable function $\hat{v} : \mathcal{S} \times \mathbb{R}^d \mapsto \mathbb{R}$

Require: a step size $\alpha > 0$

- 1: **function** approxMC($\alpha, \pi, \hat{v}$)
- 2: Initialize value-function weights $\mathbf{w} \in \mathbb{R}$ arbitrarily
- 3: **loop** forever (for each episode)
- 4: Generate an episode following π : $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$
- 5: **loop** for each step of episode $t = 0, 1, 2, \dots, T - 1$
- 6: $\mathbf{w} \leftarrow \mathbf{w} + \alpha [G_t - \hat{v}(S_t, \mathbf{w})] \nabla \hat{v}(S_t, \mathbf{w})$

Generalization and Discrimination

- Generalization: updates to one state affect the value of other states



Generalization and Discrimination

- Generalization: updates to one state affect the value of other states
 - When Link visits A, he should immediately know that B



Generalization and Discrimination

- Generalization: updates to one state affect the value of other states
 - When Link visits A, he should immediately know that B
- Discrimination: the ability to make the value of two states different



Generalization and Discrimination

- Generalization: updates to one state affect the value of other states
 - When Link visits A, he should immediately know that B
- Discrimination: the ability to make the value of two states different
 - Link should be able to differentiate the returns between A and B on the way to the treasure



Generalization and Discrimination

- Generalization: updates to one state affect the value of other states
 - When Link visits A, he should immediately know that B
- Discrimination: the ability to make the value of two states different
 - Link should be able to differentiate the returns between A and B on the way to the treasure
- Attributes here: $(x, y, \text{rupee}, \text{treasure})$

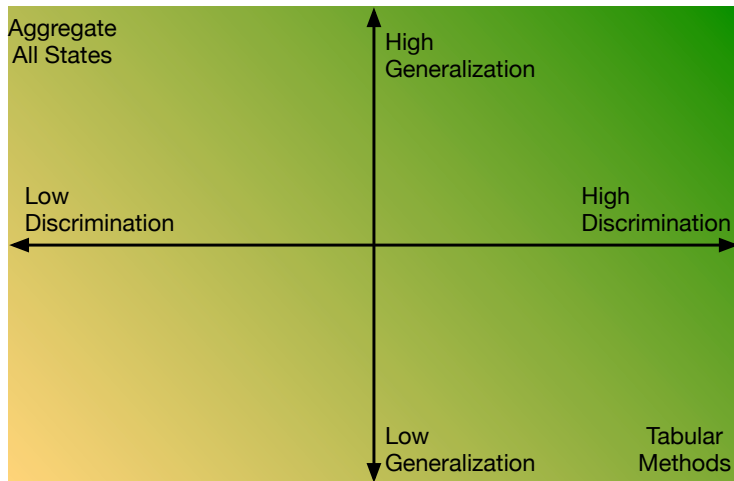


Generalization and Discrimination

- Generalization: updates to one state affect the value of other states
 - When Link visits A, he should immediately know that B
- Discrimination: the ability to make the value of two states different
 - Link should be able to differentiate the returns between A and B on the way to the treasure
- Attributes here: $(x, y, \text{rupee}, \text{treasure})$
- Some problems cannot be solved without generalization
e.g., Chess — 10^{46} states



Generalization and Discrimination



State Aggregation

State aggregation is a simple way to implement linear function approximation:

State	Value
S_1	0
S_2	0
S_3	0
S_4	0
S_5	0
S_6	0
S_7	0
S_8	0

State Aggregation

State aggregation is a simple way to implement linear function approximation:

State	Value
S_1	0
S_2	0
S_3	0
S_4	0
S_5	0
S_6	0
S_7	0
S_8	0

$$\left. \begin{array}{l} S_1 \\ S_2 \\ S_3 \\ S_4 \end{array} \right\} \mathbf{x}(s) = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$
$$\left. \begin{array}{l} S_5 \\ S_6 \\ S_7 \\ S_8 \end{array} \right\} \mathbf{x}(s) = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

State Aggregation

State aggregation is a simple way to implement linear function approximation:

State	Value
S_1	0
S_2	0
S_3	0
S_4	0
S_5	0
S_6	0
S_7	0
S_8	0

$$\left. \begin{array}{l} S_1 \\ S_2 \\ S_3 \\ S_4 \end{array} \right\} \mathbf{x}(s) = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \hat{v}(s, \mathbf{w}) = \mathbf{w}_1$$
$$\left. \begin{array}{l} S_5 \\ S_6 \\ S_7 \\ S_8 \end{array} \right\} \mathbf{x}(s) = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \quad \hat{v}(s, \mathbf{w}) = \mathbf{w}_2$$

How to Compute the Gradient for MC State Aggregation

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [\mathbf{G}_t - \hat{v}(S_t, \mathbf{w})] \nabla \hat{v}(S_t, \mathbf{w})$$

$$\nabla \hat{v}(S_t, \mathbf{w}) = \mathbf{x}(S_t) = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix}$$

How to Compute the Gradient for MC State Aggregation

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [\mathbf{G}_t - \hat{v}(S_t, \mathbf{w})] \nabla \hat{v}(S_t, \mathbf{w})$$

$$\mathbf{w}_i \leftarrow \mathbf{w}_i + \alpha [\mathbf{G}_t - \hat{v}(S_t, \mathbf{w})] 0$$

$$\nabla \hat{v}(S_t, \mathbf{w}) = \mathbf{x}(S_t) = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix}$$

How to Compute the Gradient for MC State Aggregation

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [\mathbf{G}_t - \hat{v}(S_t, \mathbf{w})] \nabla \hat{v}(S_t, \mathbf{w})$$

$$\mathbf{w}_i \leftarrow \mathbf{w}_i + \alpha [\mathbf{G}_t - \hat{v}(S_t, \mathbf{w})] 0$$

$$\mathbf{w}_j \leftarrow \mathbf{w}_j + \alpha [\mathbf{G}_t - \hat{v}(S_t, \mathbf{w})] 1$$

$$\nabla \hat{v}(S_t, \mathbf{w}) = \mathbf{x}(S_t) = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix}$$

State Aggregation in Large Random Walk

1000-state random walk environment:

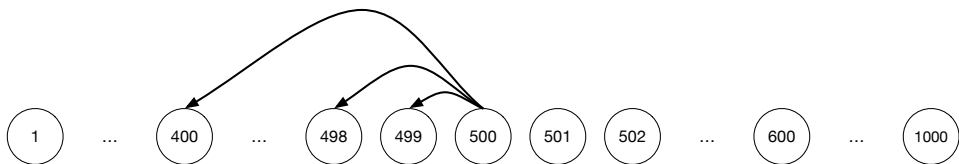
- -1 Reward on state 1
- 1 Reward on state 1000
- Left and Right action (uniformly distributed destination in 100 adjacent states)



State Aggregation in Large Random Walk

1000-state random walk environment:

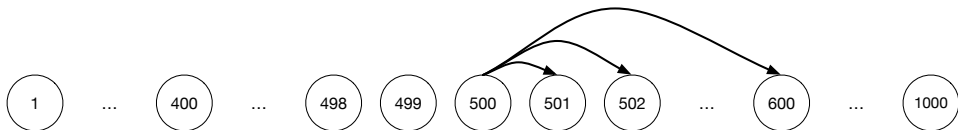
- -1 Reward on state 1
- 1 Reward on state 1000
- Left and Right action (uniformly distributed destination in 100 adjacent states)



State Aggregation in Large Random Walk

1000-state random walk environment:

- -1 Reward on state 1
- 1 Reward on state 1000
- Left and Right action (uniformly distributed destination in 100 adjacent states)

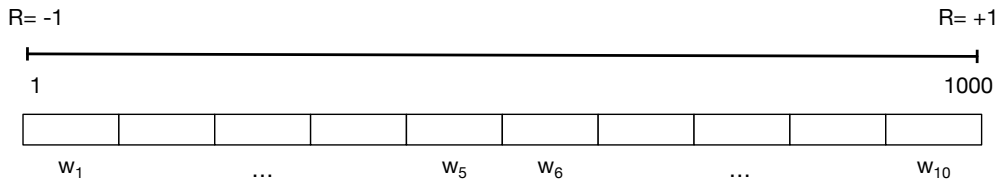


State Aggregation with Monte Carlo

Monte Carlo Updates for a Single Episode: Uniform Random Policy, $\alpha = 2 * 10^{-5}$

Return: 1, 1, 1, ..., 1

Visited: 500, 423, 483, ..., 936

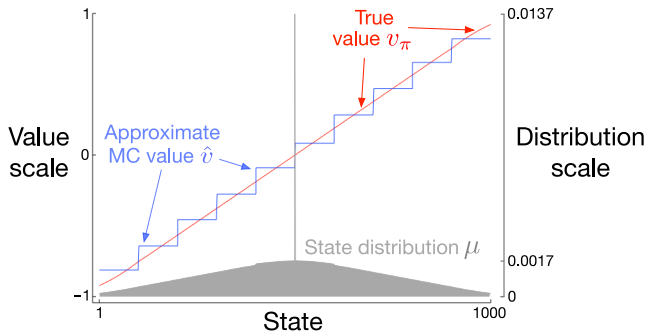


State Aggregation with Monte Carlo

Monte Carlo Updates for a Single Episode: Uniform Random Policy, $\alpha = 2 * 10^{-5}$

Return: 1, 1, 1, ..., 1

Visited: 500, 423, 483, ..., 936



TD Learning with Value Function Approximation

- The TD-target $R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w})$ is a biased sample of true value $v_\pi(s_t)$
- Can still apply supervised learning to “training data”:

$$\langle S_1, R_2 + \gamma \hat{v}(S_2, \mathbf{w}) \rangle, \langle S_2, R_3 + \gamma \hat{v}(S_3, \mathbf{w}) \rangle, \dots, \langle S_{T-1}, R_T \rangle$$

- For example, using **linear TD(0)**

$$\begin{aligned} \Delta \mathbf{w} &= \alpha (\mathbf{R} + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}) \\ &= \alpha \delta \mathbf{x}(S_t) \end{aligned}$$

- Linear TD(0) converges (close) to global optimum

The TD Update for Function Approximation

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [\textcolor{red}{U}_t - \hat{v}(S_t, \mathbf{w})] \nabla \hat{v}(S_t, \mathbf{w})$$

The TD Update for Function Approximation

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [\mathcal{U}_t - \hat{v}(S_t, \mathbf{w})] \nabla \hat{v}(S_t, \mathbf{w})$$

$$\mathcal{U}_t \doteq R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w})$$

$\mathcal{U}_t$ unbiased $\rightarrow \mathbf{w}$ will converge to a local optimum

The TD Update for Function Approximation

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [U_t - \hat{v}(S_t, \mathbf{w})] \nabla \hat{v}(S_t, \mathbf{w})$$

$$U_t \doteq R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w})$$

U_t biased $\rightarrow \mathbf{w}$ may not converge to a local optimum

TD is a **semi-gradient** method

$$\nabla \frac{1}{2} [U_t - \hat{v}(S_t, \mathbf{w})]^2$$

TD is a **semi-gradient** method

$$U_t \doteq R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w})$$

$$\begin{aligned} & \nabla \frac{1}{2} [U_t - \hat{v}(S_t, \mathbf{w})]^2 \\ &= (U_t - \hat{v}(S_t, \mathbf{w}))(\nabla U_t - \nabla \hat{v}(S_t, \mathbf{w})) \end{aligned}$$

TD is a **semi-gradient** method

$$\begin{aligned} & \nabla \frac{1}{2} [U_t - \hat{v}(S_t, \mathbf{w})]^2 \\ &= (U_t - \hat{v}(S_t, \mathbf{w}))(\nabla U_t - \nabla \hat{v}(S_t, \mathbf{w})) \neq -(U_t - \hat{v}(S_t, \mathbf{w}))\nabla \hat{v}(S_t, \mathbf{w}) \end{aligned}$$

$$\begin{aligned} U_t &\doteq R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}) \\ &\underbrace{-(U_t - \hat{v}(S_t, \mathbf{w}))\nabla \hat{v}(S_t, \mathbf{w})}_{TDUpdate} \end{aligned}$$

TD is a **semi-gradient** method

$$\begin{aligned} & \nabla \frac{1}{2} [U_t - \hat{v}(S_t, \mathbf{w})]^2 \\ &= (U_t - \hat{v}(S_t, \mathbf{w}))(\nabla U_t - \nabla \hat{v}(S_t, \mathbf{w})) \neq -(U_t - \hat{v}(S_t, \mathbf{w}))\nabla \hat{v}(S_t, \mathbf{w}) \end{aligned}$$

$$U_t \doteq R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w})$$

$$\underbrace{-(U_t - \hat{v}(S_t, \mathbf{w}))\nabla \hat{v}(S_t, \mathbf{w})}_{TDUpdate}$$

This only works if $\nabla U_t = 0$

TD is a **semi-gradient** method

$$\begin{aligned} & \nabla \frac{1}{2} [U_t - \hat{v}(S_t, \mathbf{w})]^2 \\ &= (U_t - \hat{v}(S_t, \mathbf{w}))(\nabla U_t - \nabla \hat{v}(S_t, \mathbf{w})) \neq -(U_t - \hat{v}(S_t, \mathbf{w}))\nabla \hat{v}(S_t, \mathbf{w}) \end{aligned}$$

For TD:

$$\begin{aligned} \nabla U_t &= \nabla (R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w})) \\ &= \gamma \nabla \hat{v}(S_{t+1}, \mathbf{w}) \\ &\neq 0 \end{aligned}$$

$$U_t \doteq R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w})$$

$$\underbrace{-(U_t - \hat{v}(S_t, \mathbf{w}))\nabla \hat{v}(S_t, \mathbf{w})}_{TDUpdate}$$

This only works if $\nabla U_t = 0$

Semi-Gradient TD for Policy Evaluation

Semi-gradient TD(0) for estimating $\hat{V} \approx v_\pi$

Require: the policy π to be evaluated

Require: a differentiable function $\hat{v} : \mathcal{S} \times \mathbb{R}^d \mapsto \mathbb{R}$ such that $\hat{v}(\text{terminal}, \cdot) = 0$

Require: a step size $\alpha > 0$

- 1: **function** approxTD($\alpha, \pi, \hat{v}$)
- 2: Initialize value-function weights $\mathbf{w} \in \mathbb{R}$ arbitrarily
- 3: **loop** for each episode
- 4: Initialize S
- 5: Choose $A \sim \pi(\cdot \mid S)$
- 6: Take action A, observe R, S'

until S is terminal

Semi-Gradient TD for Policy Evaluation

Semi-gradient TD(0) for estimating $\hat{V} \approx v_\pi$

Require: the policy π to be evaluated

Require: a differentiable function $\hat{v} : \mathcal{S} \times \mathbb{R}^d \mapsto \mathbb{R}$ such that $\hat{v}(\text{terminal}, \cdot) = 0$

Require: a step size $\alpha > 0$

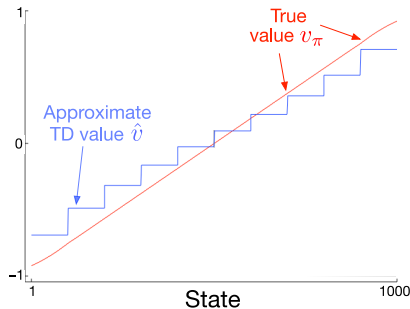
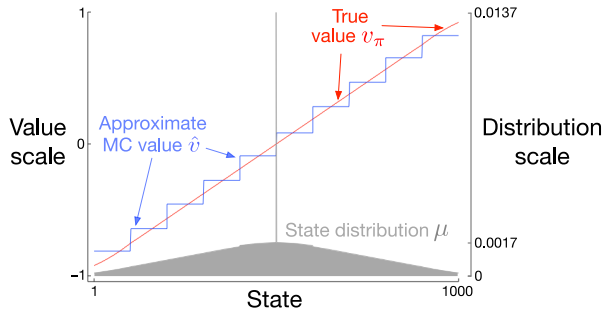
- 1: **function** approxTD($\alpha, \pi, \hat{v}$)
 - 2: Initialize value-function weights $\mathbf{w} \in \mathbb{R}$ arbitrarily
 - 3: **loop** for each episode
 - 4: Initialize S
 - 5: Choose $A \sim \pi(\cdot \mid S)$
 - 6: Take action A , observe R, S'
 - 7: $\mathbf{w} \leftarrow \mathbf{w} + \alpha [R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})] \nabla \hat{v}(S, \mathbf{w})$
 - 8: $S \leftarrow S'$
- until S is terminal

Comparing TD and Monte Carlo with State Aggregation

- The MC target uses an unbiased estimator for the returns:
thus, it is **guaranteed to converge** under certain conditions
- The TD target uses a biased estimator (i.e., the function approximation)
thus, it **may not converge**!

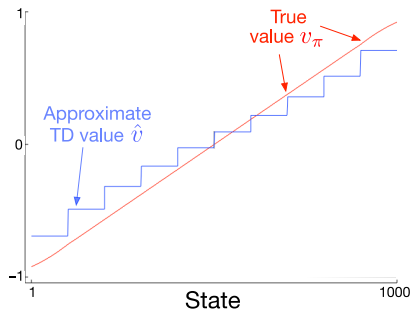
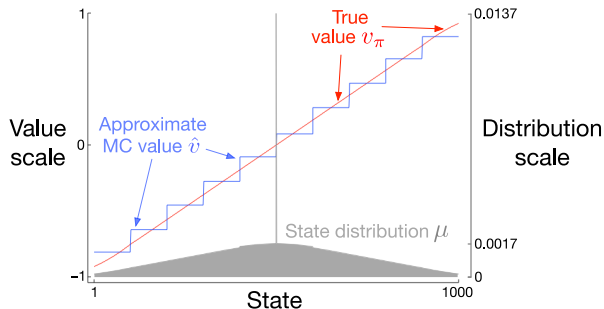
Comparing TD and Monte Carlo with State Aggregation

- The MC target uses an unbiased estimator for the returns:
thus, it is **guaranteed to converge** under certain conditions
- The TD target uses a biased estimator (i.e., the function approximation)
thus, it **may not converge**!



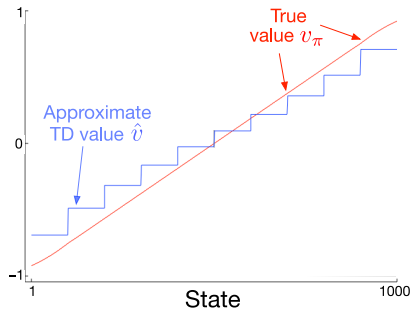
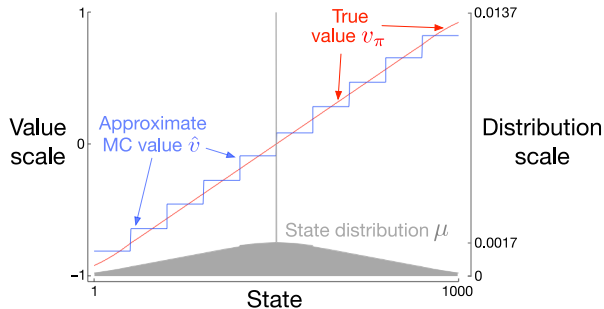
Comparing TD and Monte Carlo with State Aggregation

- The MC target uses an unbiased estimator for the returns: thus, it is **guaranteed to converge** under certain conditions
- The TD target uses a biased estimator (i.e., the function approximation) thus, it **may not converge**!
- In practice, both of them converge under linear approximation



Comparing TD and Monte Carlo with State Aggregation

- The MC target uses an unbiased estimator for the returns: thus, it is **guaranteed to converge** under certain conditions
- The TD target uses a biased estimator (i.e., the function approximation) thus, it **may not converge**!
- In practice, both of them converge under linear approximation
- However, TD learns much faster than MC



The True Objective for TD

- The True TD objective function has a fix-point for the parameters $\mathbf{w}_{\text{TD}}$,
- This fix-point is guaranteed to be smaller than the semi-gradient, depending on the discount factor:

$$\overline{VE}(\mathbf{w}_{\text{TD}}) \leq \frac{1}{1-\gamma} \min_{\mathbf{w}} \overline{VE}(\mathbf{w})$$

The True Objective for TD

- The True TD objective function has a fix-point for the parameters $\mathbf{w}_{\text{TD}}$,
- This fix-point is guaranteed to be smaller than the semi-gradient, depending on the discount factor:

$$\overline{VE}(\mathbf{w}_{\text{TD}}) \leq \frac{1}{1-\gamma} \min_{\mathbf{w}} \overline{VE}(\mathbf{w})$$

- Convergence of the semi-gradient TD approximation is more likely when $\gamma \mapsto 0$

The True Objective for TD

- The True TD objective function has a fix-point for the parameters $\mathbf{w}_{\text{TD}}$,
- This fix-point is guaranteed to be smaller than the semi-gradient, depending on the discount factor:

$$\overline{VE}(\mathbf{w}_{\text{TD}}) \leq \frac{1}{1-\gamma} \min_{\mathbf{w}} \overline{VE}(\mathbf{w})$$

- Convergence of the semi-gradient TD approximation is more likely when $\gamma \mapsto 0$
- In practice, convergence depends on the features used for linear approximation:

The True Objective for TD

- The True TD objective function has a fix-point for the parameters $\mathbf{w}_{\text{TD}}$,
- This fix-point is guaranteed to be smaller than the semi-gradient, depending on the discount factor:

$$\overline{VE}(\mathbf{w}_{\text{TD}}) \leq \frac{1}{1-\gamma} \min_{\mathbf{w}} \overline{VE}(\mathbf{w})$$

- Convergence of the semi-gradient TD approximation is more likely when $\gamma \mapsto 0$
- In practice, convergence depends on the features used for linear approximation:
 - If the features in $\mathbf{x}(\cdot)$ capture the actual state information perfectly, the TD approximation converges regardless of γ

The True Objective for TD

- The True TD objective function has a fix-point for the parameters $\mathbf{w}_{\text{TD}}$,
- This fix-point is guaranteed to be smaller than the semi-gradient, depending on the discount factor:

$$\overline{VE}(\mathbf{w}_{\text{TD}}) \leq \frac{1}{1-\gamma} \min_{\mathbf{w}} \overline{VE}(\mathbf{w})$$

- Convergence of the semi-gradient TD approximation is more likely when $\gamma \mapsto 0$
- In practice, convergence depends on the features used for linear approximation:
 - If the features in $\mathbf{x}(\cdot)$ capture the actual state information perfectly, the TD approximation converges regardless of γ
 - If the features do not capture the actual state, the error may oscilate around the true fix-point

n-Step TD Prediction with Value Function Approximation

Semi-gradient TD for estimating $\hat{V} \approx v_\pi$

Require: step size $\alpha \in (0, 1]$, small $\epsilon > 0$

Require: a differentiable function $\hat{v} : \mathcal{S} \times \mathbb{R}^d \mapsto \mathbb{R}$ such that $\hat{v}(\text{terminal}, \cdot) = 0$

Require: All store and access operations (for S_t and R_t) can take their index mod $n + 1$

```
1: function nstepApproxTDPrediction( $\alpha, \pi, n$ )
2:   Initialize value-function weights  $\mathbf{w}$  arbitrarily
3:   loop for each episode
4:     Initialize and store  $S_0 \neq \text{terminal}$ ;  $T \leftarrow \infty$ 
5:     loop for  $t = 0, 1, 2, \dots$ 
6:       if  $t < T$  then
7:         Take an action according  $\pi(\cdot \mid S_t)$ 
8:         Observe and store the next reward as  $R_{t+1}$  and state as  $S_{t+1}$ 
9:         if  $S_{t+1}$  is terminal then  $T \leftarrow t + 1$ 
10:       $\tau \leftarrow t - n + 1$   $\triangleright \tau$  is the time whose state's estimate is being updated
11:      if  $\tau \geq 0$  then
12:         $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$ 
13:        if  $\tau + n < T$  then  $G \leftarrow G + \gamma^n \hat{v}(S_{\tau+n}, \mathbf{w})$   $\triangleright G_{\tau:\tau+n}$ 
14:         $\mathbf{w} \leftarrow \mathbf{w} + \gamma [G - \hat{v}(S_\tau, \mathbf{w})] \nabla \hat{v}(S_\tau, \mathbf{w})$ 
until  $\tau = T - 1$ 
```

TD(λ) with Value Function Approximation

- The λ -return G_t^λ is a biased sample of true value $v_\pi(s_t)$
- Can again apply supervised learning to “training data”:

$$\langle S_1, G_1^\lambda \rangle, \langle S_2, G_2^\lambda \rangle, \dots, \langle S_{T-1}, G_{T-1}^\lambda \rangle$$

- Forward view linear TD(λ)

$$\begin{aligned}\Delta \mathbf{w} &= \alpha(\mathbf{G}_t^\lambda - \hat{v}(S_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}) \\ &= \alpha(\mathbf{G}_t^\lambda - \hat{v}(S_t, \mathbf{w})) \mathbf{x}(S_t)\end{aligned}$$

- Backward view linear TD(λ)

$$\begin{aligned}\delta_t &= R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}) - \hat{v}(S_t, \mathbf{w}) \\ Z_t &= \gamma \lambda Z_{t-1} + \mathbf{x}(S_t) \\ \Delta \mathbf{w} &= \alpha \delta_t Z_t\end{aligned}$$

TD(λ) with Value Function Approximation

- The λ -return G_t^λ is a biased sample of true value $v_\pi(s_t)$
- Can again apply supervised learning to “training data”:

$$\langle S_1, G_1^\lambda \rangle, \langle S_2, G_2^\lambda \rangle, \dots, \langle S_{T-1}, G_{T-1}^\lambda \rangle$$

- Forward view linear TD(λ)

$$\begin{aligned}\Delta \mathbf{w} &= \alpha(\mathbf{G}_t^\lambda - \hat{v}(S_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}) \\ &= \alpha(\mathbf{G}_t^\lambda - \hat{v}(S_t, \mathbf{w})) \mathbf{x}(S_t)\end{aligned}$$

- Backward view linear TD(λ)

$$\begin{aligned}\delta_t &= R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}) - \hat{v}(S_t, \mathbf{w}) \\ Z_t &= \gamma \lambda Z_{t-1} + \mathbf{x}(S_t) \\ \Delta \mathbf{w} &= \alpha \delta_t Z_t\end{aligned}$$

Forward view and backward view linear TD(λ) are equivalent

TD(λ) Prediction with Value Function Approximation

Semi-gradient TD for estimating $\hat{V} \approx v_\pi$

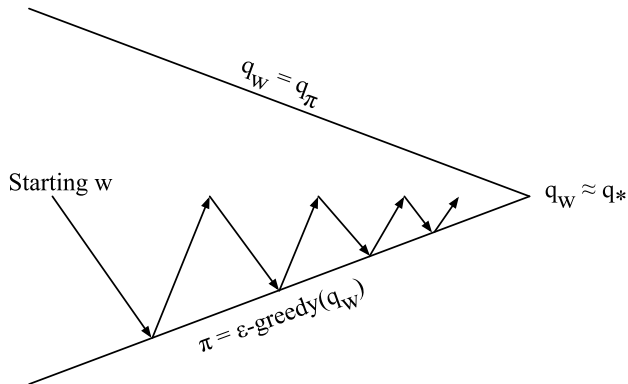
Require: the policy π to be evaluated

Require: a differentiable function $\hat{v} : \mathcal{S} \times \mathbb{R}^d \mapsto \mathbb{R}$ such that $\hat{v}(\text{terminal}, \cdot) = 0$

Require: step size $\alpha > 0$, trace decay rate $\lambda \in [0, 1]$

```
1: function lambdaApproxTDPrediction( $\alpha, \pi, \lambda$ )
2:   Initialize  $V(s) \in \mathbb{R}$  arbitrarily for all  $s \in \mathcal{S}^+$ 
3:   loop for each episode
4:     Initialize  $S$ 
5:      $Z \leftarrow 0$  ▷ a d-dimensional vector
6:     loop for each step of episode
7:       Choose  $A \sim \pi(\cdot | S)$ 
8:       Take action  $A$ , observe  $R, S'$ 
9:        $Z \leftarrow \gamma \lambda Z \nabla \hat{v}(S, \mathbf{w})$ 
10:       $\delta \leftarrow R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$ 
11:       $\mathbf{w} \leftarrow \mathbf{w} + \alpha \delta Z$ 
12:       $S \leftarrow S'$ 
   until  $S'$  is terminal
```

Control with Value Function Approximation



Policy Evaluation **Approximate** policy evaluation, $\hat{q}(\cdot, \cdot, \mathbf{w}) \approx q_\pi$

Policy Improvement ϵ -greedy policy improvement

State-values to Action-Values

$$v_{\pi}(s) \approx \hat{v}(s, \mathbf{w}) \doteq \mathbf{w}^T \mathbf{x}(s)$$

$$q_{\pi}(s, a) \approx \hat{q}(s, a, \mathbf{w}) \doteq \mathbf{w}^T \mathbf{x}(s, a)$$

Representing Actions

$$\mathbf{x}(s) = \begin{bmatrix} \mathbf{x}_0(s) \\ \mathbf{x}_1(s) \\ \mathbf{x}_2(s) \\ \mathbf{x}_3(s) \end{bmatrix}$$

$$\mathcal{A}(s) = \{\mathbf{a}_0, \mathbf{a}_1, \mathbf{a}_2\}$$

Representing Actions

$$\mathbf{x}(s) = \begin{bmatrix} \mathbf{x}_0(s) \\ \mathbf{x}_1(s) \\ \mathbf{x}_2(s) \\ \mathbf{x}_3(s) \end{bmatrix}$$

$$\mathcal{A}(s) = \{\mathbf{a}_0, \mathbf{a}_1, \mathbf{a}_2\}$$

$$\mathbf{x}(s) = \begin{bmatrix} \mathbf{x}_0(s) \\ \mathbf{x}_1(s) \\ \mathbf{x}_2(s) \\ \mathbf{x}_3(s) \\ \mathbf{x}_0(s) \\ \mathbf{x}_1(s) \\ \mathbf{x}_2(s) \\ \mathbf{x}_3(s) \\ \mathbf{x}_0(s) \\ \mathbf{x}_1(s) \\ \mathbf{x}_2(s) \\ \mathbf{x}_3(s) \end{bmatrix}$$

Representing Actions

$$\mathbf{x}(s) = \begin{bmatrix} \mathbf{x}_0(s) \\ \mathbf{x}_1(s) \\ \mathbf{x}_2(s) \\ \mathbf{x}_3(s) \end{bmatrix}$$

$$\mathcal{A}(s) = \{\mathbf{a}_0, \mathbf{a}_1, \mathbf{a}_2\}$$

$$\mathbf{x}(s) = \begin{bmatrix} \mathbf{x}_0(s) \\ \mathbf{x}_1(s) \\ \mathbf{x}_2(s) \\ \mathbf{x}_3(s) \\ \mathbf{x}_0(s) \\ \mathbf{x}_1(s) \\ \mathbf{x}_2(s) \\ \mathbf{x}_3(s) \end{bmatrix} \left. \vphantom{\begin{bmatrix} \mathbf{x}_0(s) \\ \mathbf{x}_1(s) \\ \mathbf{x}_2(s) \\ \mathbf{x}_3(s) \end{bmatrix}} \right\} \mathbf{a}_1$$

Representing Actions

$$\mathbf{x}(s) = \begin{bmatrix} \mathbf{x}_0(s) \\ \mathbf{x}_1(s) \\ \mathbf{x}_2(s) \\ \mathbf{x}_3(s) \end{bmatrix}$$

$$\mathcal{A}(s) = \{\mathbf{a}_0, \mathbf{a}_1, \mathbf{a}_2\}$$

$$\mathbf{x}(s) = \begin{bmatrix} \mathbf{x}_0(s) \\ \mathbf{x}_1(s) \\ \mathbf{x}_2(s) \\ \mathbf{x}_3(s) \\ \mathbf{x}_0(s) \\ \mathbf{x}_1(s) \\ \mathbf{x}_2(s) \\ \mathbf{x}_3(s) \\ \mathbf{x}_0(s) \\ \mathbf{x}_1(s) \\ \mathbf{x}_2(s) \\ \mathbf{x}_3(s) \end{bmatrix}$$

$$\mathbf{x}(s) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ \mathbf{x}_0(s) \\ \mathbf{x}_1(s) \\ \mathbf{x}_2(s) \\ \mathbf{x}_3(s) \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Computing action-values

$$\mathbf{x}(s_0) = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

$$\mathcal{A}(s) = \{\mathbf{a}_0, \mathbf{a}_1, \mathbf{a}_2\}$$

Computing action-values

$$\mathbf{x}(s_0) = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

$$\mathcal{A}(s) = \{a_0, a_1, a_2\}$$

$$\mathbf{w} = \begin{bmatrix} 0.7 \\ 0.1 \\ 0.4 \\ 0.4 \\ 2.2 \\ 1.0 \\ 0.6 \\ 1.8 \\ 1.3 \\ 1.1 \\ 0.9 \\ 1.7 \end{bmatrix}$$

Computing action-values

$$\mathbf{x}(s_0) = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

$$\mathcal{A}(s) = \{\mathbf{a}_0, \mathbf{a}_1, \mathbf{a}_2\}$$

$$\mathbf{w} = \begin{bmatrix} 0.7 \\ 0.1 \\ 0.4 \\ 0.4 \\ 2.2 \\ 1.0 \\ 0.6 \\ 1.8 \\ 1.3 \\ 1.1 \\ 0.9 \\ 1.7 \end{bmatrix} \quad \mathbf{x}(s_0, \mathbf{a}_0) = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$\hat{q}(s_0, \mathbf{a}_0, \mathbf{w}) = 0.7 + 0.3 = 1$$

Computing action-values

$$\mathbf{x}(s_0) = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

$$\mathcal{A}(s) = \{\mathbf{a}_0, \mathbf{a}_1, \mathbf{a}_2\}$$

$$\mathbf{w} = \begin{bmatrix} 0.7 \\ 0.1 \\ 0.4 \\ 0.4 \\ 2.2 \\ 1.0 \\ 0.6 \\ 1.8 \\ 1.3 \\ 1.1 \\ 0.9 \\ 1.7 \end{bmatrix} \quad \mathbf{x}(s_0, \mathbf{a}_0) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$\hat{q}(s_0, \mathbf{a}_0, \mathbf{w}) = 0.7 + 0.3 = 1$$

$$\hat{q}(s_0, \mathbf{a}_1, \mathbf{w}) = 2.2 + 1.8 = 4$$

$$\hat{q}(s_0, \mathbf{a}_2, \mathbf{w}) = 1.3 + 1.7 = 3$$

Computing action-values

$$\mathbf{x}(s_0) = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

$$\mathcal{A}(s) = \{\mathbf{a}_0, \mathbf{a}_1, \mathbf{a}_2\}$$

$$\mathbf{w} = \begin{bmatrix} 0.7 \\ 0.1 \\ 0.4 \\ 0.4 \\ 2.2 \\ 1.0 \\ 0.6 \\ 1.8 \\ 1.3 \\ 1.1 \\ 0.9 \\ 1.7 \end{bmatrix} \quad \mathbf{x}(s_0, \mathbf{a}_0) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

$$\hat{q}(s_0, \mathbf{a}_0, \mathbf{w}) = 0.7 + 0.3 = 1$$

$$\hat{q}(s_0, \mathbf{a}_1, \mathbf{w}) = 2.2 + 1.8 = 4$$

$$\hat{q}(s_0, \mathbf{a}_2, \mathbf{w}) = 1.3 + 1.7 = 3$$

Action-Value Function Approximation

- Approximate the action-value function

$$\hat{q}(S, A, \mathbf{w}) \approx q_{\pi}(S, A)$$

- Minimize mean-squared error between approximate action-value fn $\hat{q}(S, A, \mathbf{w})$ and true action-value fn $q_{\pi}(S, A)$

$$J(\mathbf{w}) = \mathbb{E} \left[(q_{\pi}(S, A) - \hat{q}(S, A, \mathbf{w}))^2 \right]$$

- Use stochastic gradient descent to find a local minimum

$$-\frac{1}{2} \nabla_{\mathbf{w}} J(\mathbf{w}) = (q_{\pi}(S, A) - \hat{q}(S, A, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(S, A, \mathbf{w})$$

$$\Delta \mathbf{w} = \alpha (q_{\pi}(S, A) - \hat{q}(S, A, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(S, A, \mathbf{w})$$

Linear Action-Value Function Approximation

- Represent state **and** action by a **feature vector**

$$\mathbf{x}(S, A) = \begin{pmatrix} \mathbf{x}_1(S, A) \\ \vdots \\ \mathbf{x}_n(S, A) \end{pmatrix}$$

- Represent action-value function by linear combination of features

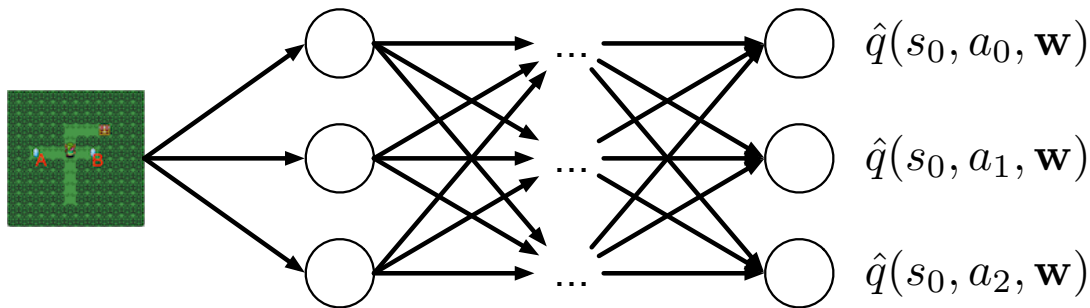
$$\hat{q}(S, A, \mathbf{w}) = \mathbf{x}(S, A)^\top \mathbf{w} = \sum_{j=1}^n \mathbf{x}_j(S, A) \mathbf{w}_j$$

- Stochastic gradient descent update

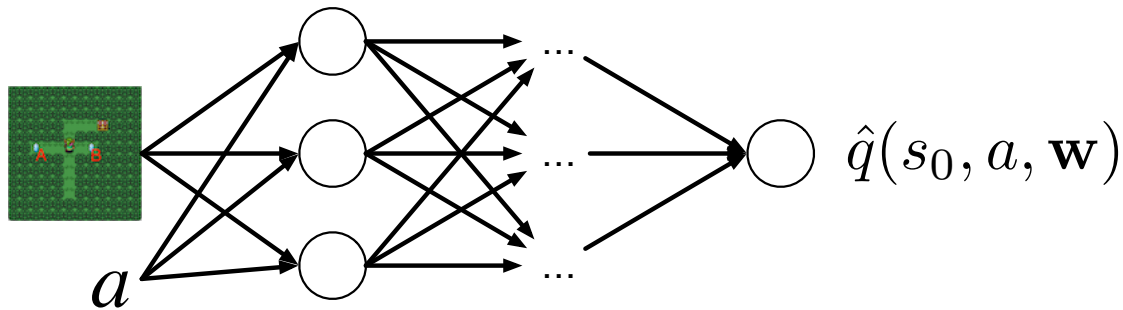
$$\nabla_{\mathbf{w}} \hat{q}(S, A, \mathbf{w}) = \mathbf{x}(S, A)$$

$$\Delta \mathbf{w} = \alpha (q_\pi(S, A) - \hat{q}(S, A, \mathbf{w})) \mathbf{x}(S, A)$$

Computing Action-values with a Neural Network



Computing Action-values with a Neural Network



Incremental Control Algorithms

- Like prediction, we must substitute a target for $q_\pi(S, A)$
 - For MC, the target is the return G_t

$$\Delta \mathbf{w} = \alpha (\mathbf{G}_t - \hat{q}(S_t, A_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(S_t, A_t, \mathbf{w})$$

- For TD(0), the target is the TD target $R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$

$$\Delta \mathbf{w} = \alpha (\mathbf{R}_{t+1} + \gamma Q(\mathbf{S}_{t+1}, \mathbf{A}_{t+1}) - \hat{q}(S_t, A_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(S_t, A_t, \mathbf{w})$$

- For forward-view TD(λ), target is the action-value λ -return

$$\Delta \mathbf{w} = \alpha (\mathbf{q}_t^\lambda - \hat{q}(S_t, A_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(S_t, A_t, \mathbf{w})$$

- For backward-view TD(λ), equivalent update is

$$\delta_t = R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w})$$

$$Z_t = \gamma \lambda Z_{t-1} + \nabla_{\mathbf{w}} \hat{q}(S_t, A_t, \mathbf{w})$$

$$\Delta \mathbf{w} = \alpha \delta_t Z_t$$

Episodic Sarsa with function approximation

Episodic Semi-gradient Sarsa for estimating $\hat{q} \approx q_*$

Require: a differentiable function $\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \mapsto \mathbb{R}$

Require: a step size $\alpha > 0$, small $\epsilon > 0$

```
1: function approxSarsa( $\alpha, \epsilon, \pi, \hat{v}$ )
2:   Initialize value-function weights  $\mathbf{w} \in \mathbb{R}$  arbitrarily
3:   loop for each episode
4:      $S, A \leftarrow$  initial state and action of episode (e.g.,  $\epsilon$ -greedy)
5:     loop for each step of episode
6:       Take action  $A$ , observe  $R, S'$ 
7:       if  $S'$  is terminal then
8:          $\mathbf{w} \leftarrow \mathbf{w} + \alpha [R - \hat{q}(S, A, \mathbf{w})] \nabla \hat{q}(S, A, \mathbf{w})$ 
9:         Go to next episode
10:      Choose  $A'$  as a function of  $\hat{q}(S', \cdot, \mathbf{w})$  (e.g.,  $\epsilon$ -greedy)
11:       $\mathbf{w} \leftarrow \mathbf{w} + \alpha [R + \gamma \hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w})] \nabla \hat{q}(S, A, \mathbf{w})$ 
12:       $S \leftarrow S'$ 
```

Expected Sarsa with Function Approximation

Sarsa:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w})] \nabla \hat{q}(S_t, A_t, \mathbf{w})$$

Expected Sarsa with Function Approximation

Sarsa:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w})] \nabla \hat{q}(S_t, A_t, \mathbf{w})$$

Expected Sarsa:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \left[R_{t+1} + \gamma \sum_{a'} \pi(a' | S_{t+1}) \hat{q}(S_{t+1}, a', \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w}) \right] \nabla \hat{q}(S_t, A_t, \mathbf{w})$$

Expected Sarsa with Function Approximation

Sarsa:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w})] \nabla \hat{q}(S_t, A_t, \mathbf{w})$$

Expected Sarsa:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \left[R_{t+1} + \gamma \sum_{a'} \pi(a' | S_{t+1}) \hat{q}(S_{t+1}, a', \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w}) \right] \nabla \hat{q}(S_t, A_t, \mathbf{w})$$

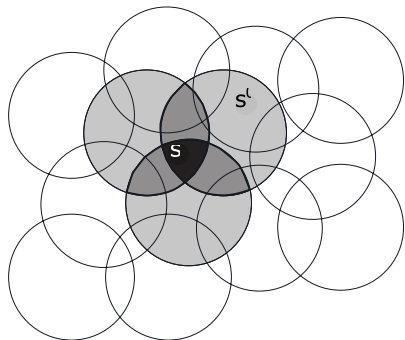
Q-Learning:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \left[R_{t+1} + \gamma \max_{a'} \hat{q}(S_{t+1}, a', \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w}) \right] \nabla \hat{q}(S_t, A_t, \mathbf{w})$$

Coarse Coding

Coarse coding: variation of state aggregation in which features **can overlap**:

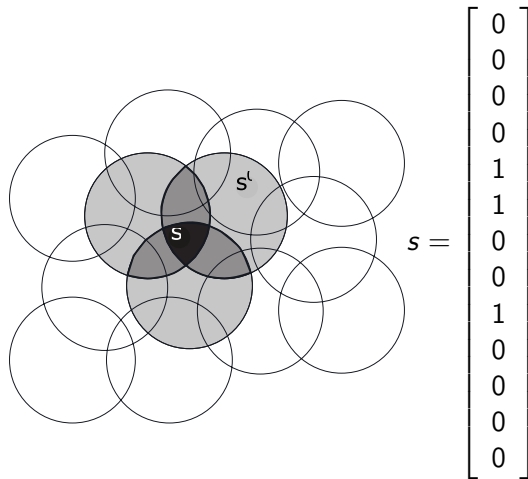
- Features can be binary or non-binary
- Provides a degree of generalization (for relatively cheap)
- Allows control over feature complexity and ultimately discrimination



Coarse Coding

Coarse coding: variation of state aggregation in which features **can overlap**:

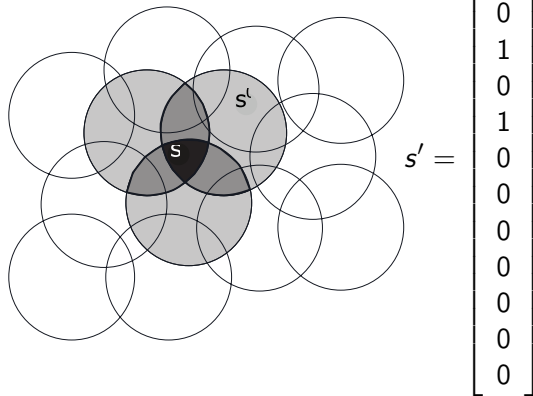
- Features can be binary or non-binary
- Provides a degree of generalization (for relatively cheap)
- Allows control over feature complexity and ultimately discrimination



Coarse Coding

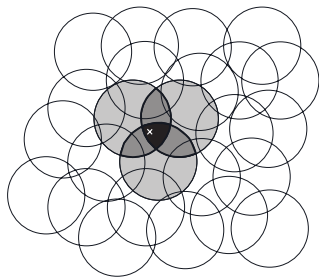
Coarse coding: variation of state aggregation in which features **can overlap**:

- Features can be binary or non-binary
- Provides a degree of generalization (for relatively cheap)
- Allows control over feature complexity and ultimately discrimination

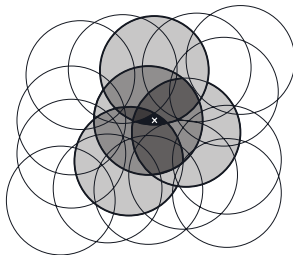


Generalization Properties of Coarse Coding

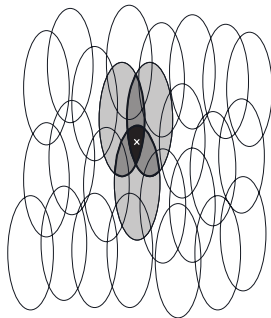
Shape, size, and number of receptive fields determine generalization and discrimination:



Narrow generalization



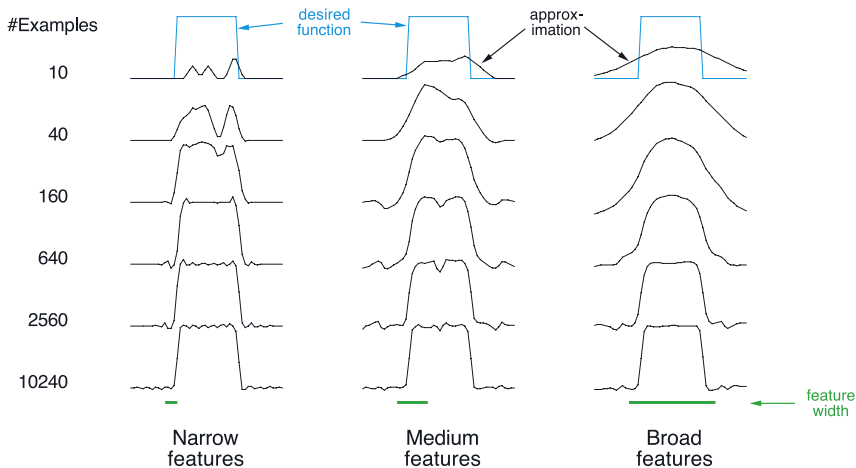
Broad generalization



Asymmetric generalization

Generalization Properties of Coarse Coding

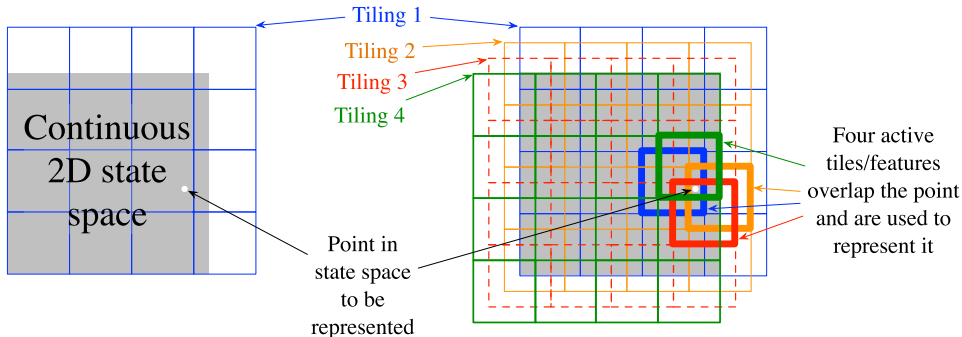
Shape, size, and number of receptive fields determine generalization and discrimination:



Tile Coding

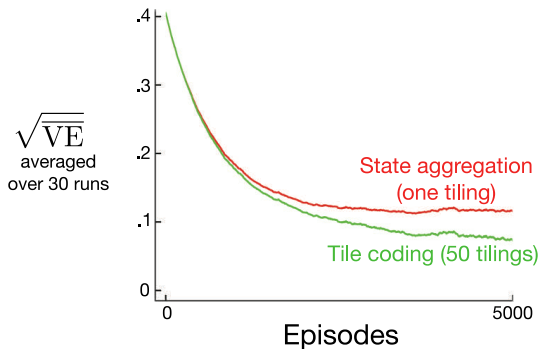
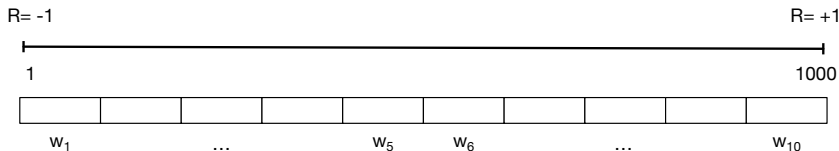
Tile coding: coarse coding for continuous multi-dimensional spaces:

- Partitions the receptive fields of the features into non-overlapping **tiles**
- Each partition is a **tiling**, often represented in a grid



Using Tile Coding in TD

Tile coding in the 1000-state Random Walk:



Non-linear Approximation with Neural Networks

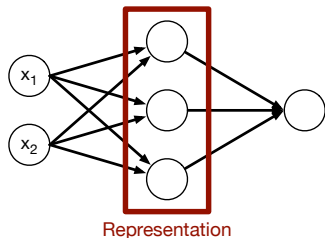
- Tile Coding representations have a number of hyperparameters:
number of tilings, tiles per tiling, offsets
- This is much like the way one parameterizes a neural network
- Neural networks are very similar to dynamically configured tile codings

$$\mathbf{x} = [x_1, x_2]^T$$

Non-linear Approximation with Neural Networks

- Tile Coding representations have a number of hyperparameters:
number of tilings, tiles per tiling, offsets
- This is much like the way one parameterizes a neural network
- Neural networks are very similar to dynamically configured tile codings

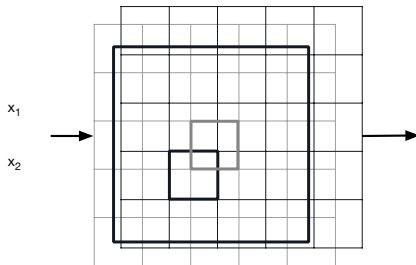
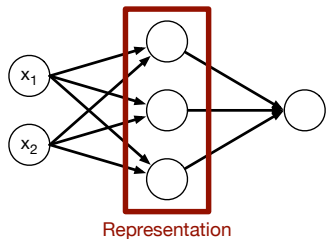
$$\mathbf{x} = [x_1, x_2]^T$$



Non-linear Approximation with Neural Networks

- Tile Coding representations have a number of hyperparameters: number of tilings, tiles per tiling, offsets
- This is much like the way one parameterizes a neural network
- Neural networks are very similar to dynamically configured tile codings

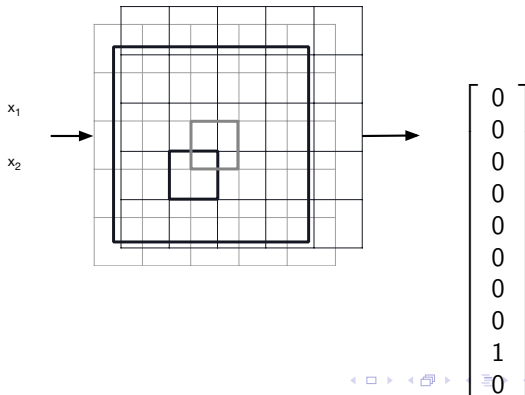
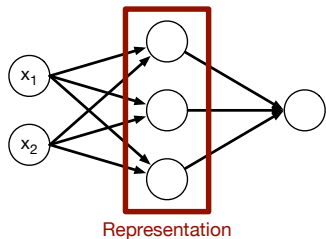
$$\mathbf{x} = [x_1, x_2]^T$$



Non-linear Approximation with Neural Networks

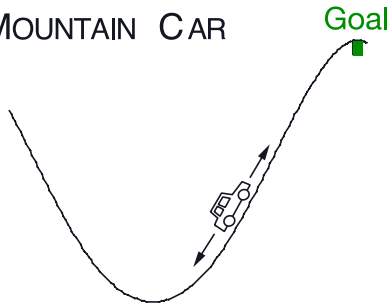
- Tile Coding representations have a number of hyperparameters: number of tilings, tiles per tiling, offsets
- This is much like the way one parameterizes a neural network
- Neural networks are very similar to dynamically configured tile codings

$$\mathbf{x} = [x_1, x_2]^T$$



The Mountain Car Environment

MOUNTAIN CAR



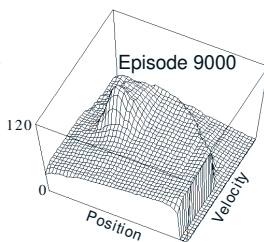
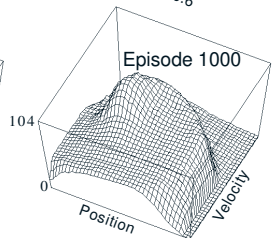
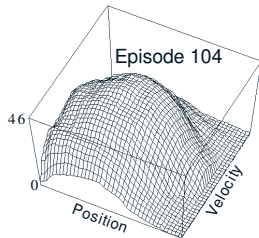
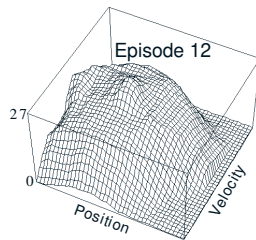
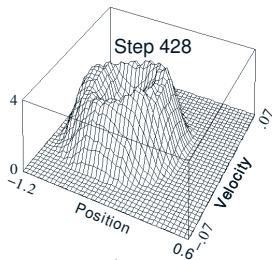
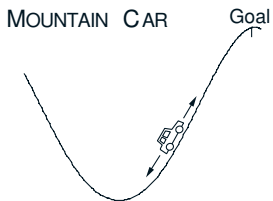
$$R_{\text{step}} = -1$$

$$\gamma = 1.0$$

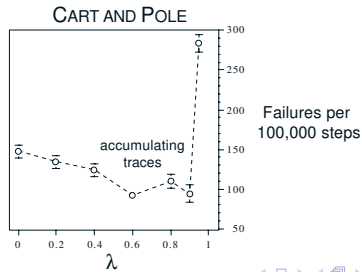
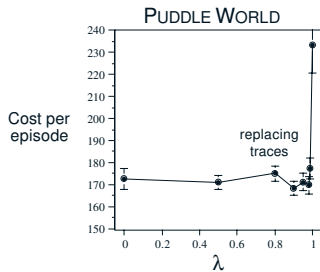
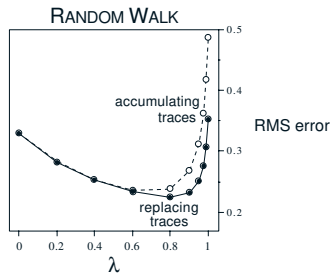
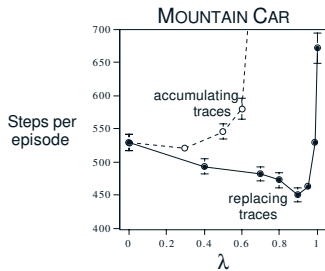
State: Car **position**
Car **velocity**

Actions: Accelerate **right**
Accelerate **left**
Coast (**no acceleration**)

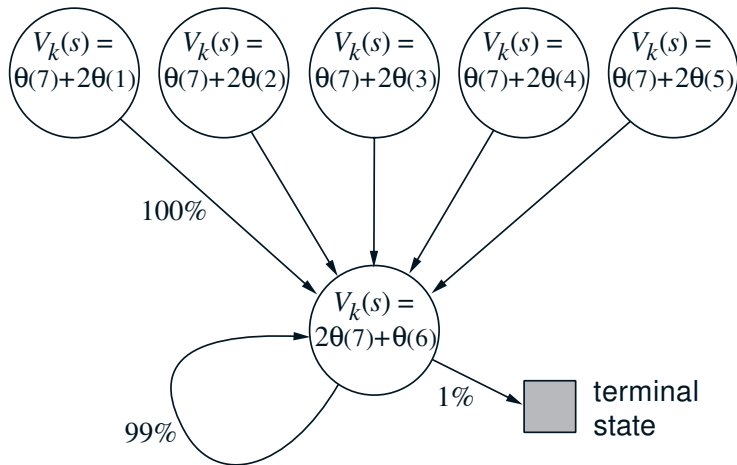
Linear Sarsa with Coarse Coding in Mountain Car



Study of λ : Should We Bootstrap?

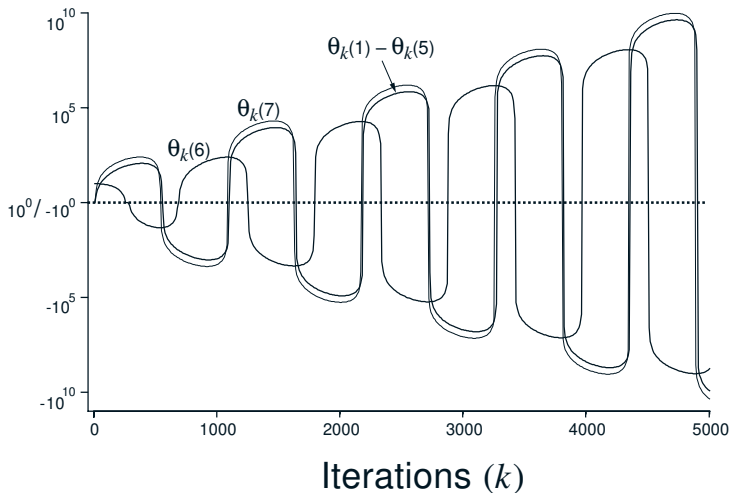


Baird's Counterexample



Parameter Divergence in Baird's Counterexample

Parameter
values, $\theta_k(i)$
(log scale,
broken at ± 1)



Convergence of Prediction Algorithms

On/Off-Policy	Algorithm	Table Lookup	Linear	Non-Linear
On-Policy	MC	✓	✓	✓
	TD(0)	✓	✓	✗
	TD(λ)	✓	✓	✗
Off-Policy	MC	✓	✓	✓
	TD(0)	✓	✗	✗
	TD(λ)	✓	✗	✗

Convergence of Prediction Algorithms

- TD does not follow the gradient of any objective function
- This is why TD can diverge when off-policy or using non-linear function approximation
- **Gradient TD** follows true gradient of projected Bellman error

On/Off-Policy	Algorithm	Table Lookup	Linear	Non-Linear
On-Policy	MC	✓	✓	✓
	TD	✓	✓	✗
	Gradient TD	✓	✓	✓
Off-Policy	MC	✓	✓	✓
	TD	✓	✗	✗
	Gradient TD	✓	✓	✓

Convergence of Control Algorithms

Algorithm	Table Lookup	Linear	Non-Linear
MC-Control	✓	(✓)	✗
SARSA	✓	(✓)	✗
Q-Learning	✓	✗	✗
Gradient Q-Learning	✓	✓	✗

(✓) = chatters around near-optimal value function

Outline

- ① Recap
- ② Introduction
- ③ Incremental Methods
 - Gradient Descent
 - Linear Function Approximation
 - Incremental Prediction Algorithms
 - Incremental Control Algorithms
 - Coarse Coding and Non-linear FA
 - Mountain Car
 - Convergence
- ④ Batch Methods
 - Least Squares Prediction
 - Least Squares Control
 - Exploration under Function Approximation
- ⑤ Summary and Conclusion

Batch Reinforcement Learning

- Gradient descent is simple and appealing
- But it is **not** sample efficient
- Batch methods seek to find the best fitting value function
- Given the agent's experience ("training data")

Least Squares Prediction

- Given value function approximation $\hat{v}(s, \mathbf{w}) \approx v(s)$
- And experience $\mathcal{D}$ consisting of $\langle \text{state}, \text{value} \rangle$ pairs

$$\mathcal{D} = \{ \langle s_1, v_1^\pi \rangle, \langle s_2, v_2^\pi \rangle, \dots, \langle s_T, v_T^\pi \rangle \}$$

- Which parameters $\mathbf{w}$ give the best fitting value function $\hat{v}(s, \mathbf{w})$?
- Least squares algorithms find parameter vector $\mathbf{w}$ minimizing sum-squared error between $\hat{v}(s_t, \mathbf{w})$ and target values v_t^π ,

$$\begin{aligned} LS(\mathbf{w}) &= \sum_{t=1}^T (v_t^\pi - \hat{v}(s_t, \mathbf{w}))^2 \\ &= \mathbb{E}_{\mathcal{D}} \left[(v_t^\pi - \hat{v}(s_t, \mathbf{w}))^2 \right] \end{aligned}$$

Stochastic Gradient Descent with Experience Replay

Given experience consisting of $\hat{v}(s, \mathbf{w})$ pairs

$$\mathcal{D} = \{\langle s_1, v_1^\pi \rangle, \langle s_2, v_2^\pi \rangle, \dots, \langle s_T, v_T^\pi \rangle\}$$

Repeat

- 1 Sample state, value from experience

$$\langle s, v^\pi \rangle \sim \mathcal{D}$$

- 2 Apply stochastic gradient descent update

$$\Delta \mathbf{w} = \alpha (v^\pi - \hat{v}(s, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w})$$

Stochastic Gradient Descent with Experience Replay

Given experience consisting of $\hat{v}(s, \mathbf{w})$ pairs

$$\mathcal{D} = \{\langle s_1, v_1^\pi \rangle, \langle s_2, v_2^\pi \rangle, \dots, \langle s_T, v_T^\pi \rangle\}$$

Repeat

- 1 Sample state, value from experience

$$\langle s, v^\pi \rangle \sim \mathcal{D}$$

- 2 Apply stochastic gradient descent update

$$\Delta \mathbf{w} = \alpha (v^\pi - \hat{v}(s, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w})$$

Converges to least squares solution

$$\mathbf{w}^\pi = \arg \min_{\mathbf{w}} LS(\mathbf{w})$$

Experience Replay in Deep Q-Networks (DQN)

DQN uses **experience replay** and **fixed Q-targets**:

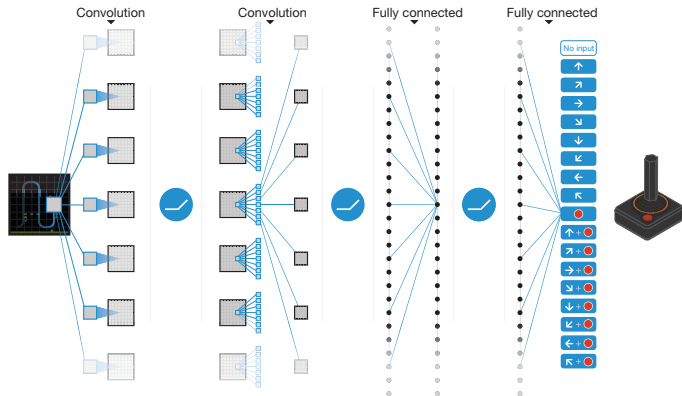
- Take action a_t according to ϵ -greedy policy
- Store transition $(s_t, a_t, r_{t+1}, s_{t+1})$ in replay memory $\mathcal{D}$
- Sample random mini-batch of transitions (s, a, r, s') from $\mathcal{D}$
- Compute Q-learning targets w.r.t. old, fixed parameters w^-
- Optimize MSE between Q-network and Q-learning targets

$$\mathcal{L}_i(w_i) = \mathbb{E}_{s,a,r,s' \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; w_i^-) - Q(s, a; w_i) \right)^2 \right]$$

- Using variant of stochastic gradient descent

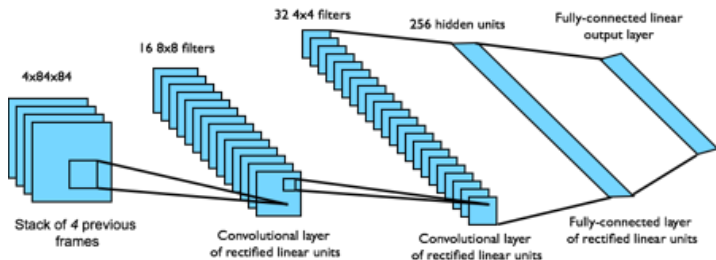
DQN in Atari

- End-to-end learning of values $Q(s, a)$ from pixels s
- Input state s is stack of raw pixels from last 4 frames
- Output is $Q(s, a)$ for 18 joystick/button positions
- Reward is change in score for that step

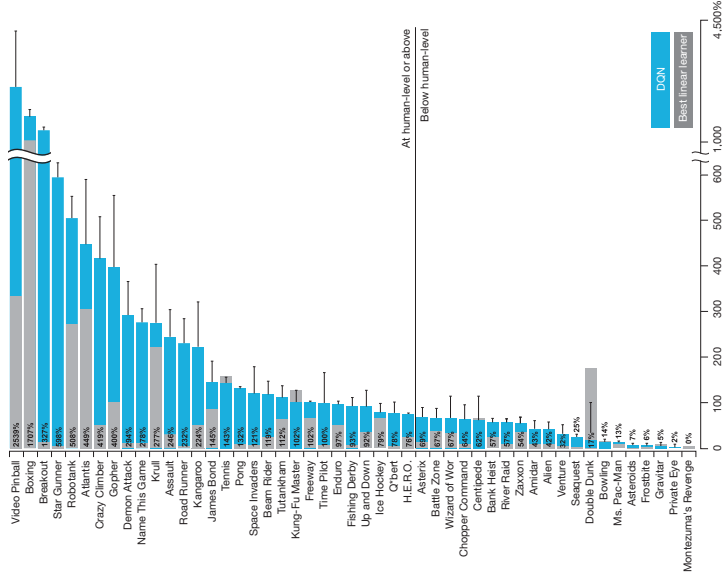


DQN in Atari

- End-to-end learning of values $Q(s, a)$ from pixels s
- Input state s is stack of raw pixels from last 4 frames
- Output is $Q(s, a)$ for 18 joystick/button positions
- Reward is change in score for that step



DQN Results in Atari



How much does DQN help?

	Replay Fixed-Q	Replay Q-learning	No replay Fixed-Q	No replay Q-learning			BrFS	UCT
Breakout	316.81	240.73	10.16	3.17	384	772	1	364
Enduro	1006.3	831.25	141.89	29.1	500	359	1	286
River Raid	7446.62	4102.81	2867.66	1453.02	5694	5062	2179	4449
Seaquest	2894.4	822.55	1003	275.81	14272	6138	288	5132
Space Invaders	1088.94	826.33	373.22	301.99	2877	3974	112	2718

Refs: Mnih, V., Kavukcuoglu, K., Silver, D. et al. Human-level control through deep reinforcement learning. Nature 518, 529–533 (2015).

How much does DQN help?

	Replay Fixed-Q	Replay Q-learning	No replay Fixed-Q	No replay Q-learning	No ML IW(1)	No ML 2BFS	BrFS	UCT
Breakout	316.81	240.73	10.16	3.17	384	772	1	364
Enduro	1006.3	831.25	141.89	29.1	500	359	1	286
River Raid	7446.62	4102.81	2867.66	1453.02	5694	5062	2179	4449
Seaquest	2894.4	822.55	1003	275.81	14272	6138	288	5132
Space Invaders	1088.94	826.33	373.22	301.99	2877	3974	112	2718

Refs: Mnih, V., Kavukcuoglu, K., Silver, D. et al. Human-level control through deep reinforcement learning. Nature 518, 529–533 (2015).

Lipovetzky, Nir, Ramírez, Miguel, and Geffner, Hector. Classical Planning with Simulators: Results on the Atari Video Games. IJCAI (2015).

Linear Least Squares Prediction

- Experience replay finds least squares solution
- But it may take many iterations
- Using linear value function approximation $\hat{v}(s, \mathbf{w}) = \mathbf{x}(s)^\top \mathbf{w}$
- We can solve the least squares solution directly

Linear Least Squares Prediction (2)

- At minimum of $LS(\mathbf{w})$, the expected update must be zero

$$\mathbb{E}_{\mathcal{D}} [\Delta \mathbf{w}] = 0$$

$$\alpha \sum_{t=1}^T \mathbf{x}(s_t)(v_t^{\pi} - \mathbf{x}(s_t)^{\top} \mathbf{w}) = 0$$

$$\sum_{t=1}^T \mathbf{x}(s_t)v_t^{\pi} = \sum_{t=1}^T \mathbf{x}(s_t)\mathbf{x}(s_t)^{\top} \mathbf{w}$$

$$\mathbf{w} = \left(\sum_{t=1}^T \mathbf{x}(s_t)\mathbf{x}(s_t)^{\top} \right)^{-1} \sum_{t=1}^T \mathbf{x}(s_t)v_t^{\pi}$$

- For N features, direct solution time is $O(N^3)$
- Incremental solution time is $O(N^2)$ using Sherman-Morrison

Linear Least Squares Prediction Algorithms

- We do not know true values v_t^π
- In practice, our “training data” must use noisy or biased samples of v_t^π

LSMC Least Squares Monte-Carlo uses return

$$v_t^\pi \approx G_t$$

LSTD Least Squares Temporal-Difference uses TD target

$$v_t^\pi \approx R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w})$$

LSTD(λ) Least Squares TD(λ) uses λ -return

$$v_t^\pi \approx G_t^\lambda$$

- In each case solve directly for fixed point of MC / TD / TD(λ)

Linear Least Squares Prediction Algorithms

LSMC

$$0 = \sum_{t=1}^T \alpha (G_t - \hat{v}(S_t, \mathbf{w})) \mathbf{x}(S_t)$$

$$\mathbf{w} = \left(\sum_{t=1}^T \mathbf{x}(S_t) \mathbf{x}(S_t)^\top \right)^{-1} \sum_{t=1}^T \mathbf{x}(S_t) G_t$$

LSTD

$$0 = \sum_{t=1}^T \alpha (R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}) - \hat{v}(S_t, \mathbf{w})) \mathbf{x}(S_t)$$

$$\mathbf{w} = \left(\sum_{t=1}^T \mathbf{x}(S_t) (\mathbf{x}(S_t) - \gamma \mathbf{x}(S_{t+1}))^\top \right)^{-1} \sum_{t=1}^T \mathbf{x}(S_t) R_{t+1}$$

LSTD(λ)

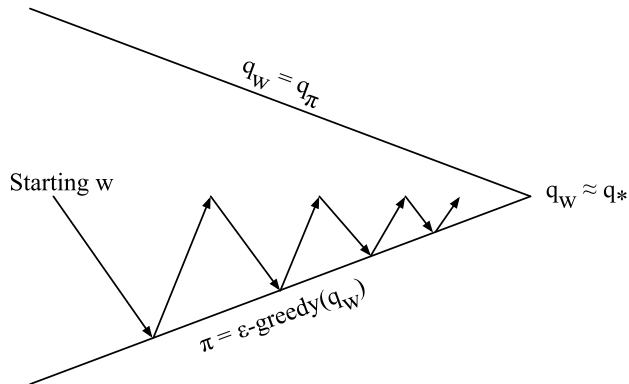
$$0 = \sum_{t=1}^T \alpha \delta_t Z_t$$

$$\mathbf{w} = \left(\sum_{t=1}^T Z_t (\mathbf{x}(S_t) - \gamma \mathbf{x}(S_{t+1}))^\top \right)^{-1} \sum_{t=1}^T Z_t R_{t+1}$$

Convergence of Linear Least Squares Prediction Algorithms

On/Off-Policy	Algorithm	Table Lookup	Linear	Non-Linear
On-Policy	MC	✓	✓	✓
	LSMC	✓	✓	-
	TD	✓	✓	✗
	LSTD	✓	✓	-
Off-Policy	MC	✓	✓	✓
	LSMC	✓	✓	-
	TD	✓	✗	✗
	LSTD	✓	✗	-

Least Squares Policy Iteration



Policy Evaluation Policy evaluation by **least squares Q-learning**
Policy Improvement Greedy policy improvement

Least Squares Action-Value Function Approximation

- Approximate action-value function $q_\pi(s, a)$ using linear combination of features $\mathbf{x}(s, a)$

$$\hat{q}(s, a, \mathbf{w}) = \mathbf{x}(s, a)^\top \mathbf{w} \approx q_\pi(s, a)$$

- Minimize least squares error between $\hat{q}(s, a, \mathbf{w})$ and $q_\pi(s, a)$ from experience generated using policy π consisting of $\langle (state, action), value \rangle$ pairs

$$\mathcal{D} = \{ \langle (s_1, a_1), v_1^\pi \rangle, \langle (s_2, a_2), v_2^\pi \rangle, \dots, \langle (s_T, a_T), v_T^\pi \rangle \}$$

Least Squares Control

- For policy evaluation, we want to efficiently use all experience
- For control, we also want to improve the policy
- This experience is generated from many policies
- So to evaluate $q_\pi(S, A)$ we must learn off-policy
- We use the same idea as Q-learning:
 - Use experience generated by old policy
 $S_t, A_t, R_{t+1}, S_{t+1} \sim \pi_{old}$
 - Consider alternative successor action $A' = \pi_{new}(S_{t+1})$
 - Update $\hat{q}(S_t, A_t, \mathbf{w})$ towards value of alternative action $R_{t+1} + \gamma \hat{q}(S_{t+1}, A', \mathbf{w})$

Least Squares Q-Learning

- Consider the following linear Q-learning update

$$\begin{aligned}\delta &= R_{t+1} + \gamma \hat{q}(S_{t+1}, \pi(S_{t+1}), \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w}) \\ \Delta \mathbf{w} &= \alpha \delta \mathbf{x}(S_t, A_t)\end{aligned}$$

- LSTDQ algorithm: solve for total update = zero

$$\begin{aligned}0 &= \sum_{t=1}^T \alpha (R_{t+1} + \gamma \hat{q}(S_{t+1}, \pi(S_{t+1}), \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w})) \mathbf{x}(S_t, A_t) \\ \mathbf{w} &= \left(\sum_{t=1}^T \mathbf{x}(S_t, A_t) (\mathbf{x}(S_t, A_t) - \gamma \mathbf{x}(S_{t+1}, \pi(S_{t+1})))^\top \right)^{-1} \sum_{t=1}^T \mathbf{x}(S_t, A_t) R_{t+1}\end{aligned}$$

Least Squares Policy Iteration Algorithm

- The following pseudocode uses LSTDQ for policy evaluation
- It repeatedly re-evaluates experience $\mathcal{D}$ with different policies

Least Squares Policy Iteration Algorithm

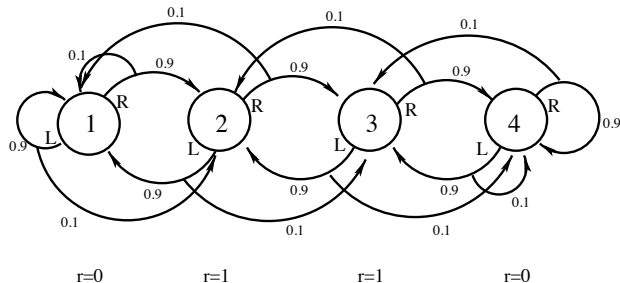
```
1: function LSPI-TD( $\mathcal{D}, \pi_0$ )
2:    $\pi' \leftarrow \pi_0$ 
3:   repeat
4:      $\pi \leftarrow \pi'$ 
5:      $Q \leftarrow \text{LSTDQ}(\pi, \mathcal{D})$ 
6:     for all  $s \in \mathcal{S}$  do
7:        $\pi'(s) \leftarrow \arg \max_{a \in \mathcal{A}} Q(s, a)$ 
8:   until  $\pi \approx \pi'$ 
9:   return  $\pi$ 
```

Convergence of Control Algorithms

Algorithm	Table Lookup	Linear	Non-Linear
MC-Control	✓	(✓)	✗
SARSA	✓	(✓)	✗
Q-Learning	✓	✗	✗
LSPI	✓	(✓)	-

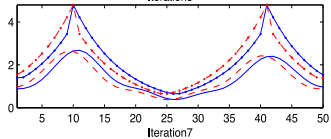
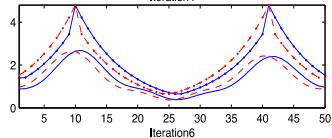
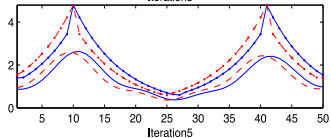
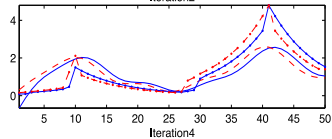
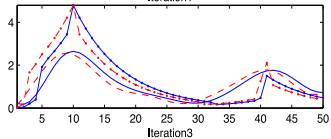
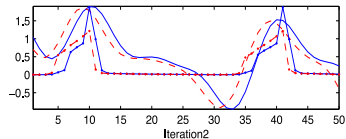
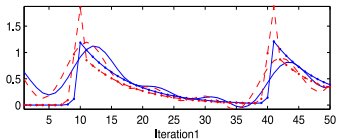
(✓) = chatters around near-optimal value function

Chain Walk Example

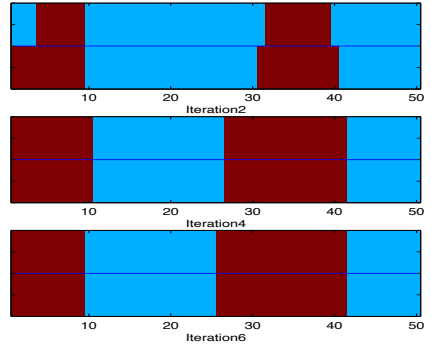
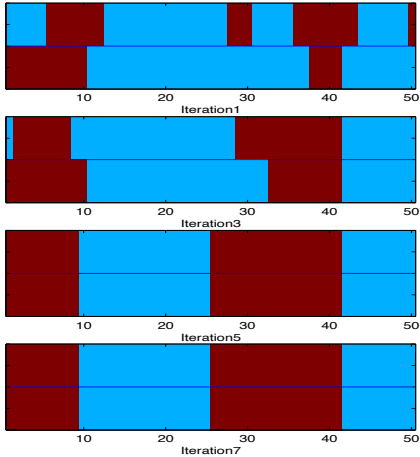


- Consider the 50 state version of this problem
- Reward +1 in states 10 and 41, 0 elsewhere
- Optimal policy: R (1-9), L (10-25), R (26-41), L (42, 50)
- Features: 10 evenly spaced Gaussians ($\sigma = 4$) for each action
- Experience: 10,000 steps from random walk policy

LSPI in Chain Walk: Action-Value Function



LSPI in Chain Walk: Policy



Optimistic Initialization

- Optimistic initialization in the tabular setting is trivial
simply initialize the maximum possible value to all states: $q(s, a) \leftarrow 1000 \quad \forall s, a$
- However, in function approximation, this is not trivial

Optimistic Initialization

- Optimistic initialization in the tabular setting is trivial
simply initialize the maximum possible value to all states: $q(s, a) \leftarrow 1000 \quad \forall s, a$
- However, in function approximation, this is not trivial

Linear:

$$q_{\pi}(s, a) \approx \hat{q}(s, a, \mathbf{w}) = \mathbf{w}^T \mathbf{x}(s, a)$$

Optimistic Initialization

- Optimistic initialization in the tabular setting is trivial
simply initialize the maximum possible value to all states: $q(s, a) \leftarrow 1000 \quad \forall s, a$
- However, in function approximation, this is not trivial

Linear:

$$q_{\pi}(s, a) \approx \hat{q}(s, a, \mathbf{w}) = \mathbf{w}^T \mathbf{x}(s, a)$$

$$\mathbf{w} \leftarrow \begin{bmatrix} 100 \\ 100 \\ 100 \\ \vdots \end{bmatrix}$$

Optimistic Initialization

- Optimistic initialization in the tabular setting is trivial
simply initialize the maximum possible value to all states: $q(s, a) \leftarrow 1000 \quad \forall s, a$
- However, in function approximation, this is not trivial

Linear:

$$q_{\pi}(s, a) \approx \hat{q}(s, a, \mathbf{w}) = \mathbf{w}^T \mathbf{x}(s, a)$$

$$\mathbf{w} \leftarrow \begin{bmatrix} 100 \\ 100 \\ 100 \\ \vdots \end{bmatrix}$$

Non-linear:

$$q_{\pi}(s, a) \approx \hat{q}(s, a, \mathbf{w}) = \text{NN}(s, a, \mathbf{w})$$

$$\mathbf{w} \leftarrow ???$$

ϵ -greedy

In practice, we will often use ϵ -greedy exploration when using function approximation

$$1 - \epsilon$$

$$A_t = \arg \max_a \hat{q}(S_t, a, \mathbf{w})$$

$$\epsilon$$

$$A_t = \text{Random}(\mathcal{A})$$

Outline

- ① Recap
- ② Introduction
- ③ Incremental Methods
 - Gradient Descent
 - Linear Function Approximation
 - Incremental Prediction Algorithms
 - Incremental Control Algorithms
 - Coarse Coding and Non-linear FA
 - Mountain Car
 - Convergence
- ④ Batch Methods
 - Least Squares Prediction
 - Least Squares Control
 - Exploration under Function Approximation
- ⑤ Summary and Conclusion

Lecture Summary

- Function Approximation in Reinforcement Learning
 - Value function representations
 - Reducing MC to supervised learning
 - Incremental methods for function approximation and gradient descent
Incremental prediction and control algorithms
 - Linear and Nonlinear function approximation
Coarse coding and Neural Networks
 - Batch methods and exploration in function approximation